



UNIVERSITAT
POLITÈCNICA
DE VALÈNCIA

Aprendizaje Automático Probabilístico

Redes neuronales para datos tabulares

Alfons Juan

DSIC

Departamento de Sistemas
Informáticos y Computación

Índice

13.0 Resumen	1
13.1 Introducción	7
13.2 Perceptrones multicapa	9
13.2.1 El problema XOR	10
13.2.2 MLPs diferenciables	12
13.2.3 Funciones de activación	14
13.2.4 Modelos ejemplo	17
13.2.4.1 MLP para clasificar datos 2d en 2 categorías	17
13.2.4.2 MLP para clasificación de imágenes	18
13.2.4.3 MLP para clasificación de texto	20
13.2.4.4 MLP para regresión heterocedástica	22
13.2.5 La importancia de la profundidad	24
13.2.6 La “revolución del aprendizaje profundo”	26
13.2.7 Conexiones con la biología	27
13.3 Backpropagation	31
13.3.1 Diferenciación hacia adelante y hacia atrás	32
13.3.2 Diferenciación hacia atrás para MLPs	39

13.3.3	Productos vector-Jacobiana para las capas	43
13.3.3.1	Capa de entropía cruzada	44
13.3.3.2	Capa de no-linealidad elemental	46
13.3.3.3	Capa lineal	47
13.3.4	Grafos de computación	50
13.4	Entrenamiento de redes neuronales	54
13.4.1	Ajuste del factor de aprendizaje	55
13.4.2	Gradientes que desaparecen y explotan	56
13.4.3	Funciones de activación no saturantes	59
13.4.3.1	ReLU	63
13.4.3.2	ReLU no saturante	64
13.4.3.3	Otras opciones	65
13.4.4	Conexiones residuales	66
13.4.5	Inicialización de parámetros	69
13.4.5.1	Esquemas de inicialización heurísticos	69
13.4.5.2	Inicializaciones dirigidas por los datos	71
13.4.5.3	Interpretación como espacio funcional	72
13.4.6	Entrenamiento paralelo	78
13.5	Regularización	80

13.5.1	Terminación temprana	80
13.5.2	Penalización de pesos	80
13.5.3	DNNs dispersas	80
13.5.4	Dropout	84
13.5.5	Redes neuronales Bayesianas	86

13.0. Resumen

► *Introducción:*

- ▷ Expansión de funciones base: flexibilizamos modelos lineales reemplazando la entrada con funciones base parametrizadas y no prefijadas
- ▷ **FFNN** o **MLP**: composición de L funciones o capas para datos tabulares
- ▷ DNN: composición de funciones diferenciables que transforma una entrada (datos tabulares o no) en una salida con un grafo dirigido acíclico

- ▶ **Perceptrones multicapa:** partimos de Perceptrón, versión determinista de regresión logística, $f(x; \theta) = \mathbb{1}(\mathbf{w}^t \mathbf{x} + b \geq 0) = H(\mathbf{w}^t \mathbf{x} + b)$
- ▷ **El problema XOR:** no resoluble por Perceptrón
- ▷ **MLPs diferenciables:** composición de Perceptrones con funciones de activación diferenciables (en lugar de H)
- ▷ **Funciones de activación:** no lineales como σ , $\tanh$ o ReLU
- ▷ **Modelos ejemplo:** para datos 2d en 2 categorías, clasificación de imágenes, clasificación de texto y regresión heterocedástica
- ▷ **La importancia de la profundidad:** aunque basta una capa oculta (con suficientes unidades) para aproximar toda función (mínimamente suave), las profundas ofrecen mejores resultados
- ▷ **La “revolución del aprendizaje profundo”:** 2010- (ASR y CV) con GPUs, muchos datos y librerías de código abierto
- ▷ **Conexiones con la biología:** inspirados en el modelo neuronal de McCulloch-Pitts, en realidad son modelos “simples”

► **Backpropagation:** calcula el gradiente de una función de pérdida aplicada a la salida de una red con respecto a los parámetros de cada capa; se pasa a un algoritmo de optimización

▷ **Diferenciación hacia adelante y hacia atrás:** $f : \mathbb{R}^n \rightarrow \mathbb{R}^m$

↳ **Jacobiana:**

$$f = f_4 \circ f_3 \circ f_2 \circ f_1$$

$$x \rightarrow x_2 = f_1(x) \rightarrow x_3 = f_2(x_2) \rightarrow x_4 = f_3(x_3) \rightarrow o = f_4(x_4)$$

$$J_f(x) = \frac{\partial o}{\partial x} = J_{f_4}(x_4) J_{f_3}(x_3) J_{f_2}(x_2) J_{f_1}(x) \in \mathbb{R}^{m \times n}$$

↳ **Diferenciación hacia adelante:** si $n \leq m$, con n JVPs

$$J_f(x)v = \underbrace{J_{f_4}(x_4)}_{m \times m_3} \underbrace{J_{f_3}(x_3)}_{m_3 \times m_2} \underbrace{J_{f_2}(x_2)}_{m_2 \times m_1} \underbrace{J_{f_1}(x)}_{m_1 \times n} \underbrace{v}_{n \times 1}$$

↳ **Diferenciación hacia atrás:** si $m \leq n$, con m VJPs

$$u^t J_f(x) = \underbrace{u^t}_{1 \times m} \underbrace{J_{f_4}(x_4)}_{m \times m_3} \underbrace{J_{f_3}(x_3)}_{m_3 \times m_2} \underbrace{J_{f_2}(x_2)}_{m_2 \times m_1} \underbrace{J_{f_1}(x)}_{m_1 \times n}$$

▷ **Diferenciación hacia atrás para MLPs: backprop!**

↳ **Forward:** $x_1 = x$ $x_{k+1} = f_k(x_k, \theta_k)$ para $k = 1 : K$

↳ **Backward:** $u_{K+1} = 1$ $g_k = u_{k+1}^t \frac{\partial f_k(x_k, \theta_k)}{\partial \theta_k}$ $u_k = u_{k+1}^t \frac{\partial f_k(x_k, \theta_k)}{\partial x_k}$

↳ **Salida:** $\mathcal{L} = x_{K+1}$ $\nabla_x \mathcal{L} = u_1$ $\{\nabla_{\theta_k} \mathcal{L} = g_k\}_{k=1}^K$

▷ **Productos vector-Jacobiana para las capas:**

↳ **Capa de entropía cruzada:** $z = -\sum_c y_c \log p_c$ $\mathbf{J} = (\mathbf{p} - \mathbf{y})^t$

↳ **Capa de no-linealidad elemental:** $z = \varphi(x)$ $\mathbf{J} = \text{diag}(\varphi'(x))$

↳ **Capa lineal:** $z = \mathbf{W}x$ $\mathbf{J} = \mathbf{W}$

▷ **Grafos de computación:**

↳ **Estáticos:** calculados en preproceso

↳ **Dinámicos:** calculados “just in time”

- ▶ **Entrenamiento de redes neuronales:** minimización de la NLL (regularizada) con SGD (Adam) y gradientes backprop

$$\mathcal{L}(\boldsymbol{\theta}) = -\log p(\mathcal{D} \mid \boldsymbol{\theta}) = -\sum_{n=1}^N \log p(\mathbf{y}_n \mid \mathbf{x}_n; \boldsymbol{\theta})$$

- ▷ **Ajuste del factor de aprendizaje:** necesario con SGD
- ▷ **Gradientes que desaparecen y explotan:** gradient clipping para los que explotan; para los que desaparecen ...
- ▷ **Funciones de activación no saturantes:** leaky ReLU, ELU, Swish, GELU
- ▷ **Conexiones residuales:** capa residual $\mathcal{F}'_l = \mathcal{F}_l(\mathbf{x}) + \mathbf{x}$
- ▷ **Inicialización de parámetros:** Xavier o Glorot, LeCun, He, uniforme o dirigidas por los datos
- ▷ **Entrenamiento paralelo:** paralelismo de datos, multi-GPU

► **Regularización**

- ▷ **Terminación temprana:** el método más sencillo
- ▷ **Penalización de pesos:** regularización ℓ_2
- ▷ **DNNs dispersas:** ℓ_1 , ARD y otros métodos; poco usadas
- ▷ **Dropout:** apagado aleatorio de cada neurona; muy popular
- ▷ **Redes neuronales Bayesianas:** ensamble de redes (costoso)

13.1. Introducció

► *Expansió de funcions base:*

- Flexibilizamos modelos lineales reemplazando la entrada x con funciones base $\phi(x)$:

$$f(x; \theta) = \mathbf{W} \phi(x) + b \quad (1)$$

con $\theta = (\mathbf{W}, b)$

- Pero no fijamos el extractor de características $\phi(x)$ a mano, sino que lo parametrizamos con su propio θ_2 (a aprender):

$$f(x; \theta) = \mathbf{W} \phi(x; \theta_2) + b \quad (2)$$

con $\theta = (\theta_1, \theta_2)$ y $\theta_1 = (\mathbf{W}, b)$

- **Feedforward neural network (FFNN) o multilayer perceptron (MLP):** compone L funciones o capas $f_\ell(\mathbf{x}; \boldsymbol{\theta}_\ell)$ análogamente,

$$f(\mathbf{x}; \boldsymbol{\theta}) = f_L(f_{L-1}(\cdots f_1(\mathbf{x}) \cdots)) \quad (3)$$

asumiendo **datos estructurados o tabulares**, $\mathbf{x} \in \mathbb{R}^D$

- **Deep neural network (DNN):** composición de funciones diferenciables que transforma una entrada estructurada o no (imagen, secuencia o grafo) en una salida con un grafo dirigido acíclico

13.2. Perceptrones multicapa

- **Perceptrón:** versión determinista de regresión logística,

$$f(\mathbf{x}; \boldsymbol{\theta}) = \mathbb{1}(\mathbf{w}^t \mathbf{x} + b \geq 0) = H(\mathbf{w}^t \mathbf{x} + b) \quad (4)$$

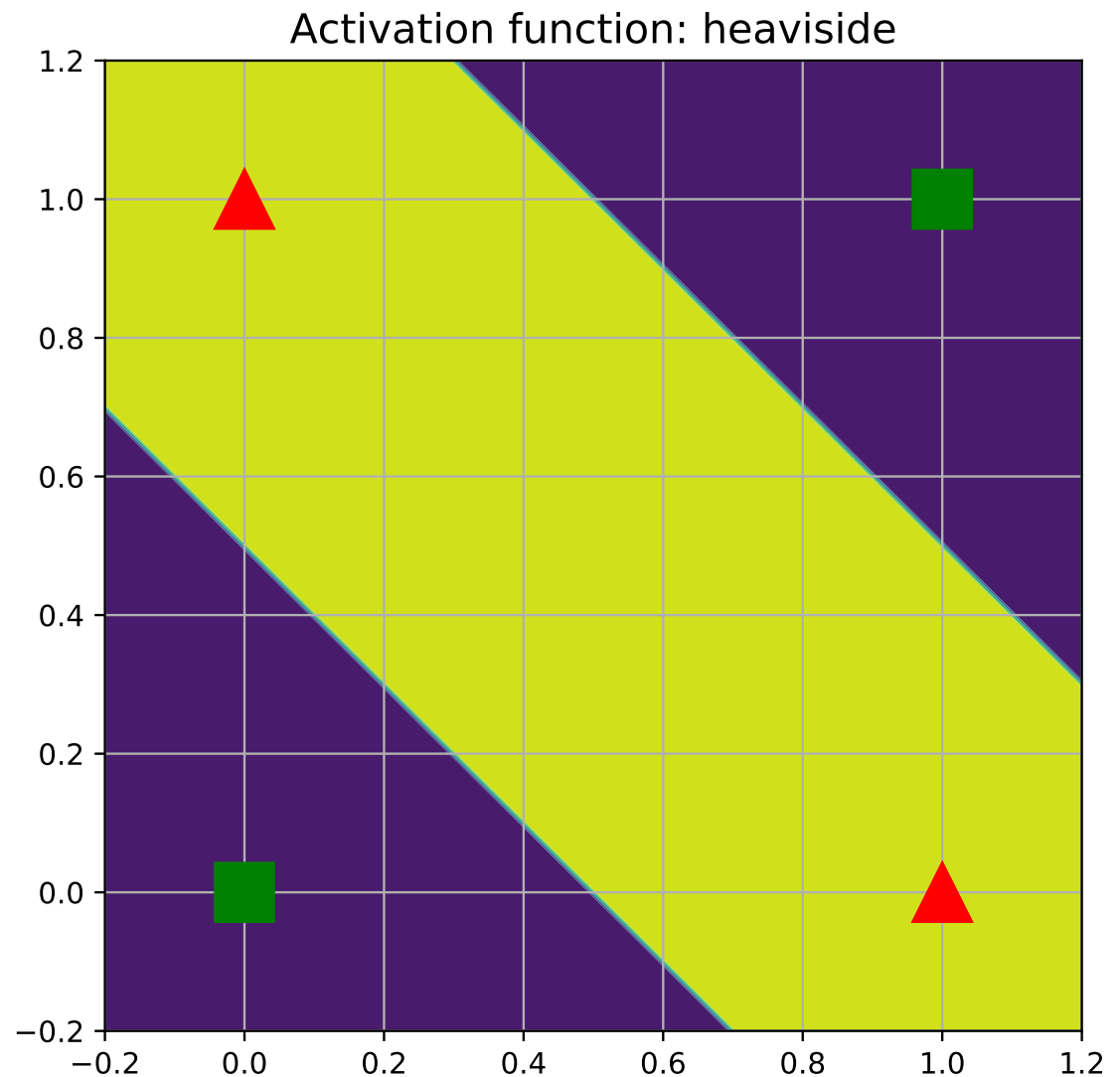
donde $H(a)$ es la función escalón de Heaviside o umbral lineal

- **Libro “Perceptrons”:** publicado en 1969 por Marvin Minsky y Seymour Papert, proporcionó numerosos ejemplos de problemas de reconocimiento de formas que los perceptrones no resuelven

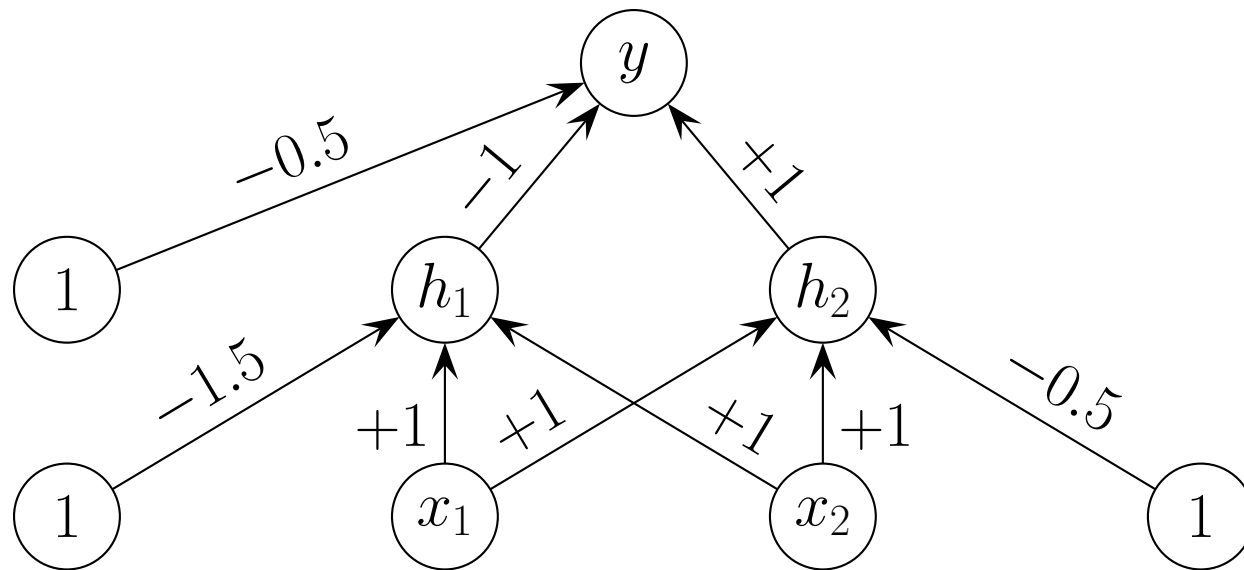
13.2.1. El problema XOR

- *Problema XOR*: problema famoso del libro “Perceptrons” con datos no linealmente separables (mediante un Perceptrón)

x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	0



- Resolución de XOR: MLP compuesto de 3 Perceptrones, correspondientes a dos *unidades ocultas*, h_1 y h_2 , y la salida y



x_1	x_2	$\mathbf{w}_1^t \mathbf{x} + b_1$	h_1	$\mathbf{w}_2^t \mathbf{x} + b_2$	h_2	$\mathbf{w}_y^t \mathbf{h} + b_y$	y
0	0	-1.5	0	-0.5	0	-0.5	0
0	1	-0.5	0	+0.5	1	+0.5	1
1	0	-0.5	0	+0.5	1	+0.5	1
1	1	+0.5	1	+1.5	1	-0.5	0

$$y = \neg h_1 \wedge h_2 = \neg(x_1 \wedge x_2) \wedge (x_1 \vee x_2)$$

13.2.2. MLPs diferenciables

- **Función de activación:** función diferenciable

$$\varphi : \mathbb{R} \rightarrow \mathbb{R} \quad (5)$$

- La función escalón de Heaviside, H , no es diferenciable

- **Unidades ocultas de la capa ℓ , $\mathbf{z}_\ell$:** se definen como una transformación lineal de las unidades ocultas de la capa previa transformadas elementalmente por la función de activación,

$$\mathbf{z}_\ell = f_\ell(\mathbf{z}_{\ell-1}) = \varphi_\ell(\mathbf{b}_\ell + \mathbf{W}_\ell \mathbf{z}_{\ell-1}) \quad (6)$$

o, en forma escalar,

$$z_{k\ell} = \varphi_\ell \left(b_{k\ell} + \sum_{j=1}^{K_{\ell-1}} w_{jkl} z_{j\ell-1} \right) \quad (7)$$

► *Pre-activaciones* de la capa ℓ :

$$\mathbf{a}_\ell = \mathbf{b}_\ell + \mathbf{W}_\ell \mathbf{z}_{\ell-1} \quad (8)$$

► *MLPs (diferenciables):*

- ▷ Son composiciones de capas con funciones diferenciables
- ▷ Mucho más usados que los MLPs no diferenciables históricos (con función escalón de Heaviside)

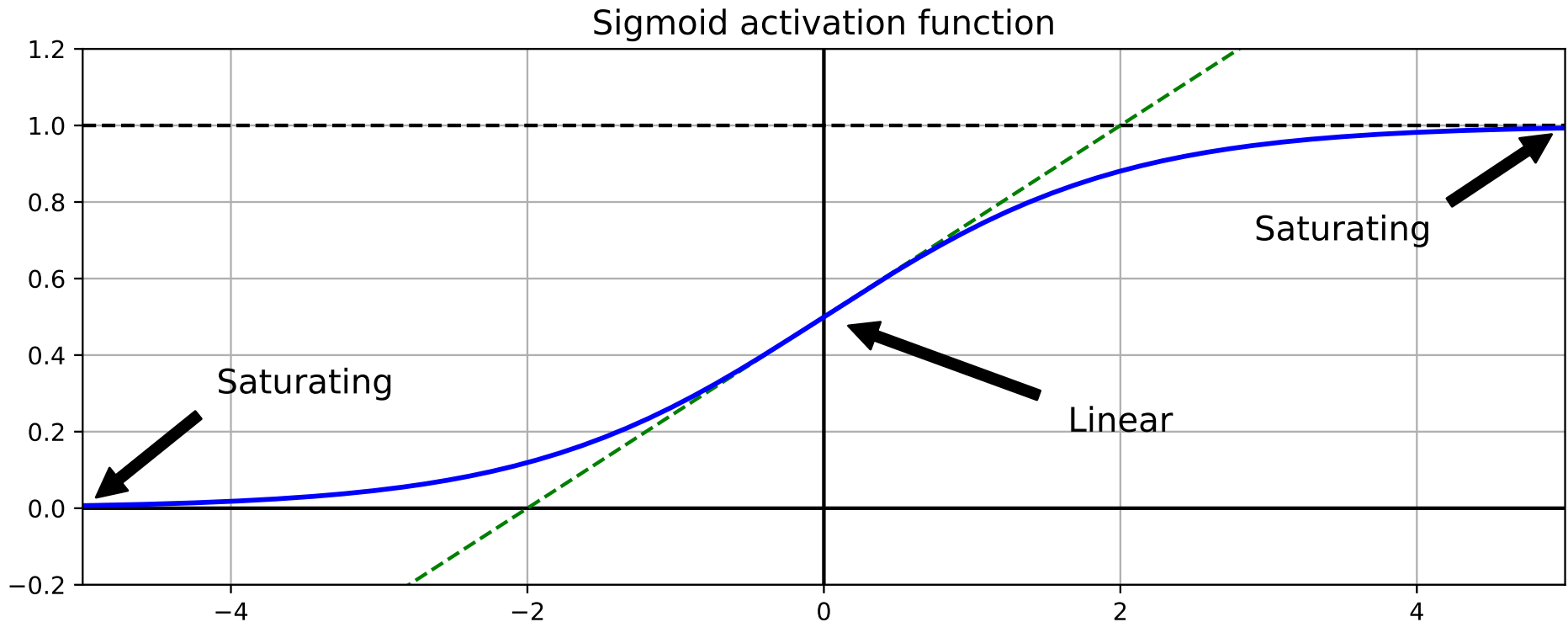
13.2.3. Funciones de activación

► **Lineales:** si $\varphi(a) = c_\ell a$, el modelo completo es lineal

$$\begin{aligned} f(\mathbf{x}; \boldsymbol{\theta}) &= \mathbf{W}_L c_L(\mathbf{W}_{L-1} c_{L-1}(\cdots (\mathbf{W}_1 \mathbf{x}) \cdots)) \\ &\propto \tilde{\mathbf{W}}_L \tilde{\mathbf{W}}_{L-1} \cdots \tilde{\mathbf{W}}_1 \mathbf{x} \\ &= \tilde{\mathbf{W}} \mathbf{x} \end{aligned}$$

► Toda composición de capas lineales sucesivas se reduce a una sola capa lineal

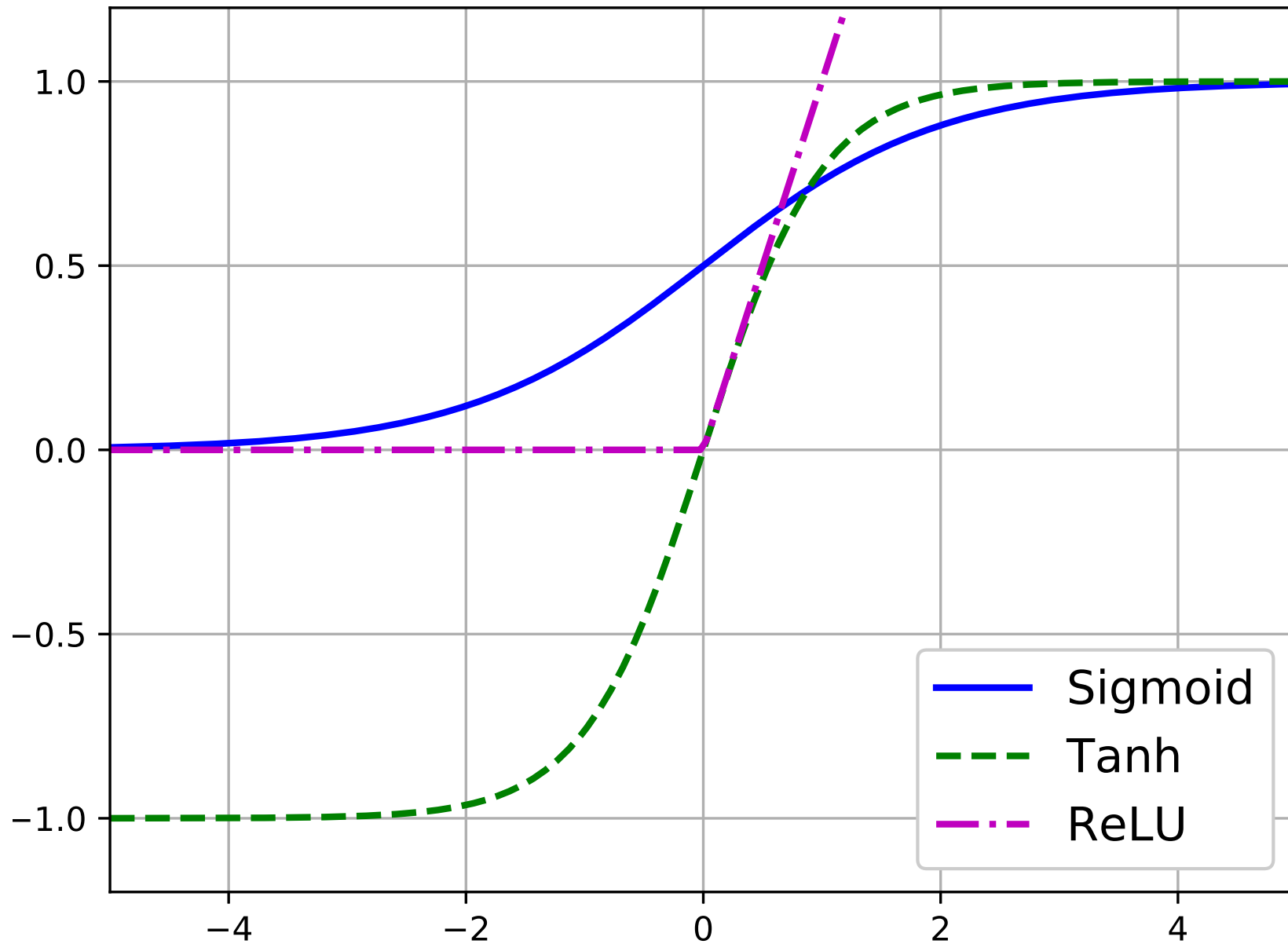
- **Funciones sigmoide (logística) y tanh:** antes populares, **saturan** a $(0, 1)$ y $(-1, 1)$, lo que origina gradientes muy pequeños



- **No saturantes: ReLU (rectified linear unit)** muy popular

$$\text{ReLU}(a) = \max(a, 0) = a \mathbb{1}(a > 0) \quad (9)$$

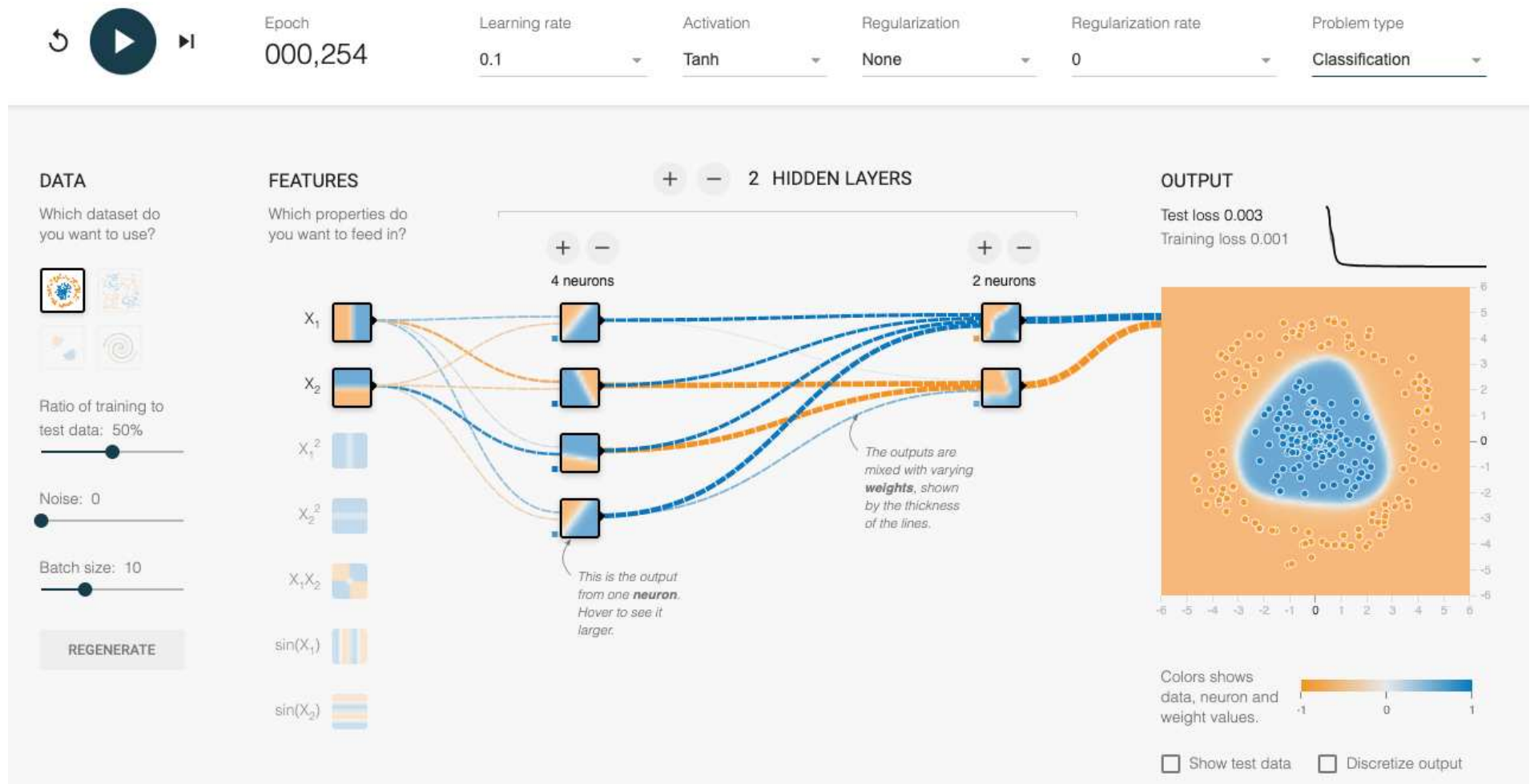
Activation functions



13.2.4. Modelos ejemplo

13.2.4.1. MLP para clasificar datos 2d en 2 categorías

► <http://playground.tensorflow.org>



13.2.4.2. MLP para clasificación de imágenes

- *Aplanado (flatten)* necesario de la imagen 2d en un vector 1d
- MLP para MNIST: entrada $28 \times 28 = 784$ dimensional, dos capas ocultas de 128 unidades cada una y capa softmax con 10 salidas

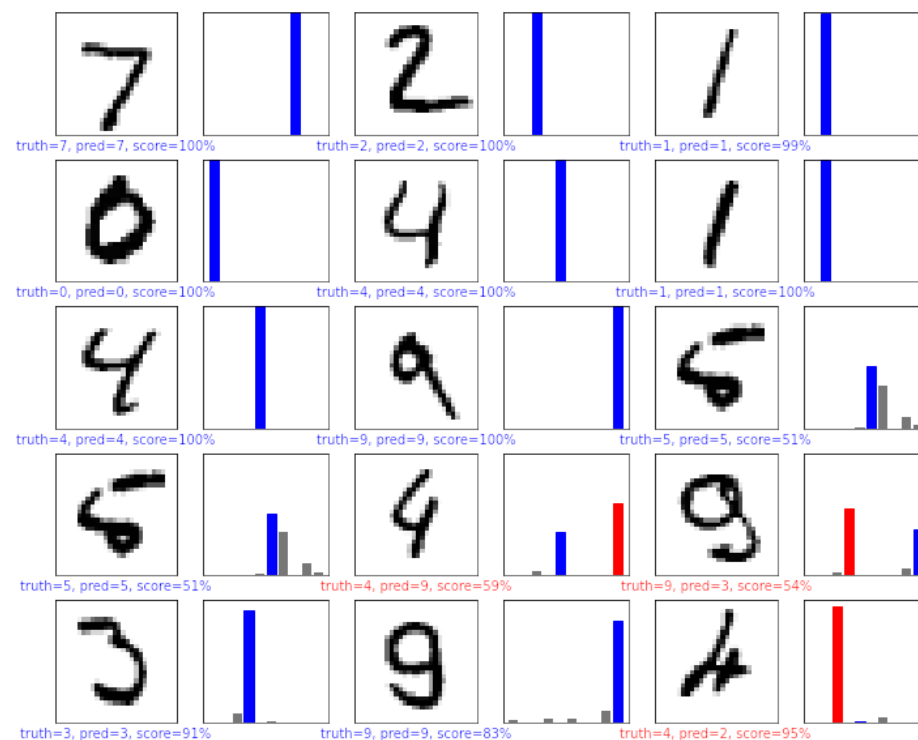
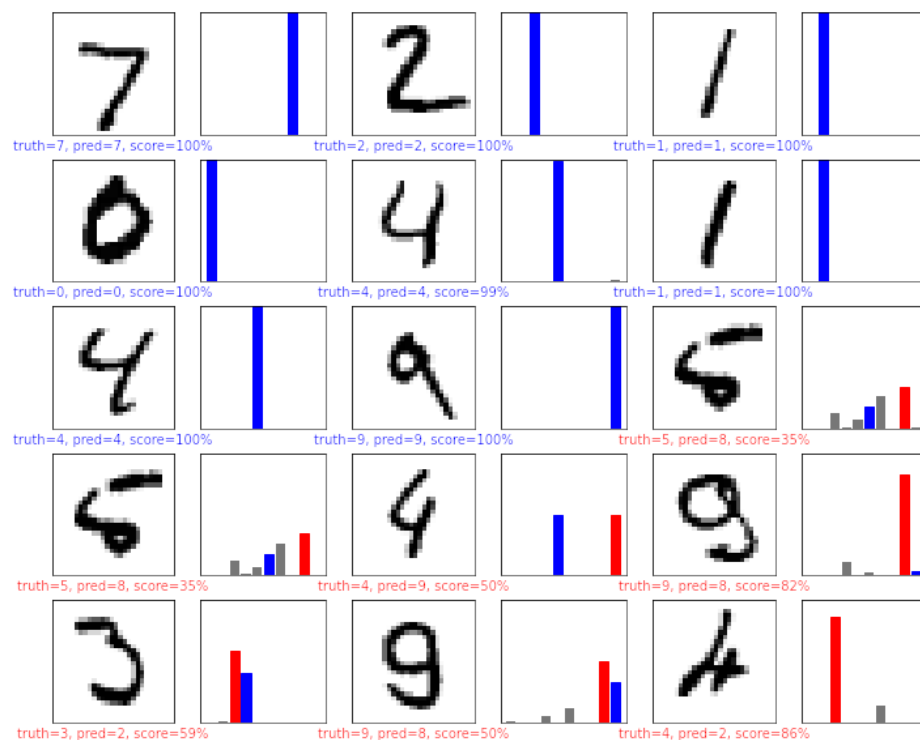
```
Model: "sequential"
```

Layer (type)	Output Shape	Param #
flatten (Flatten)	(None, 784)	0
dense (Dense)	(None, 128)	100480
dense_1 (Dense)	(None, 128)	16512
dense_2 (Dense)	(None, 10)	1290
Total params: 118,282		
Trainable params: 118,282		
Non-trainable params: 0		

$$100\,840 = (784 + 1) \cdot 128$$

$$16\,512 = (128 + 1) \cdot 128$$

- 15 ejemplos de clasificación (escogidos para incluir errores)
 - ▷ Ejemplos tras una (izq.) y dos iteraciones (dcha.) de aprendizaje
 - ▷ Azul es correcta; rojo (y gris) incorrecta
 - ▷ Tras dos iteraciones, la precisión de test es del 97.1 %



13.2.4.3. MLP para clasificación de texto

- ▶ La entrada es una secuencia de palabras de longitud variable, $v_1, \dots, v_T$, siendo v_t un vector one-hot de dimensión igual a la talla del vocabulario, V ; la tratamos como bolsa de palabras, $\{v_t\}$
- ▶ La primera capa es una matriz de encaje $E \times V$, $\mathbf{W}_1$, que convierte cada vector disperso v_t en uno denso e_t ,

$$e_t = \mathbf{W}_1 v_t \quad (10)$$

- ▶ La segunda capa convierte la entrada en un único vector E -dimensional mediante *global average pooling*,

$$\bar{e} = \frac{1}{T} \sum_{t=1}^T e_t \quad (11)$$

- ▶ La salida de la segunda capa se pasa a un MLP; por ejemplo, con una capa oculta y salida logística binaria:

$$\mathbf{h} = \varphi(\mathbf{W}_2 \bar{e} + \mathbf{b}_2) \quad (12)$$

$$p(y \mid \mathbf{x}; \boldsymbol{\theta}) = \text{Ber}(y \mid \boldsymbol{\sigma}(\mathbf{w}_3^t \mathbf{h} + b_3)) \quad (13)$$

- MLP para IMDB: vocabulario de talla $V = 10000$, encaje de dimensión $E = 16$ y capa oculta de 16 unidades

Model: "sequential"

Layer (type)	Output Shape	Param #
embedding (Embedding)	(None, None, 16)	160000
global_average_pooling1d (GlobalAveragePooling1D)	(None, 16)	0
dense (Dense)	(None, 16)	272
dense_1 (Dense)	(None, 1)	17
Total params: 160,289		
Trainable params: 160,289		
Non-trainable params: 0		

$$160\,000 = 10\,000 \cdot 16 \quad 272 = (16 + 1) \cdot 16$$

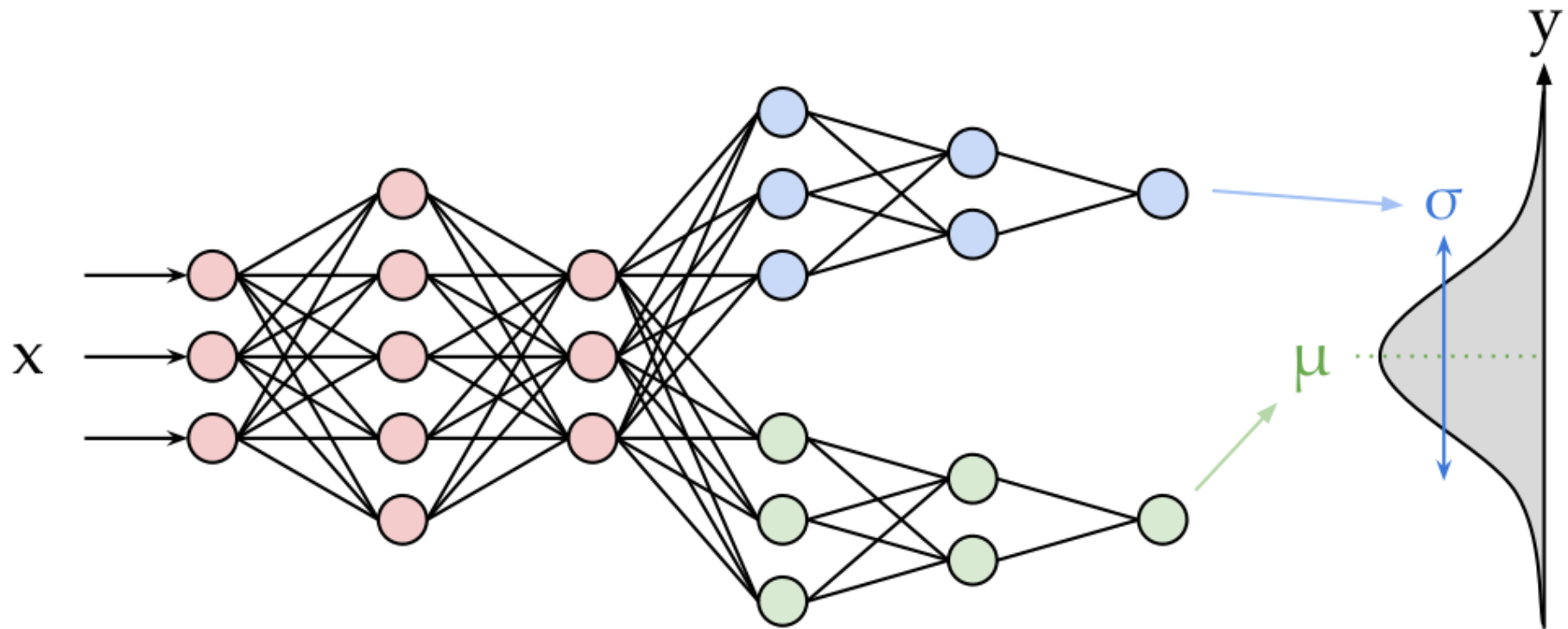
Precisión en validación del 86 %

13.2.4.4. MLP para regresión heterocedástica

- MLP con *backbone* común y dos *heads* para hallar μ y σ

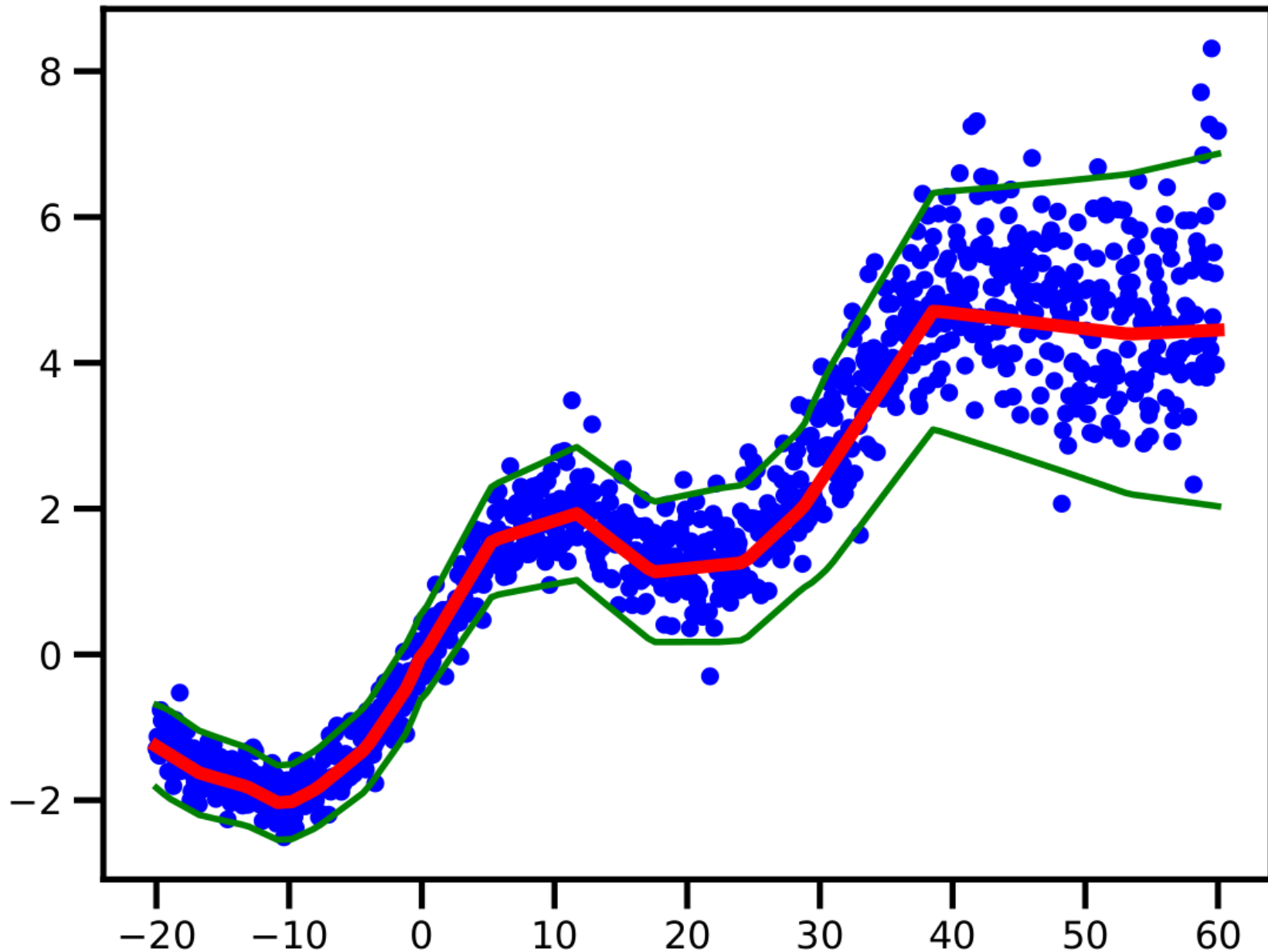
$$f_{\mu}(\mathbf{x}) = \mathbb{E}[y \mid \mathbf{x}, \boldsymbol{\theta}] \quad \text{con} \quad \varphi(a) = a \quad (14)$$

$$f_{\sigma}(\mathbf{x}) = \sqrt{\mathbb{V}[y \mid \mathbf{x}, \boldsymbol{\theta}]} \quad \text{con} \quad \varphi(a) = \sigma_{+}(a) = \log(1 + e^a) \quad (15)$$



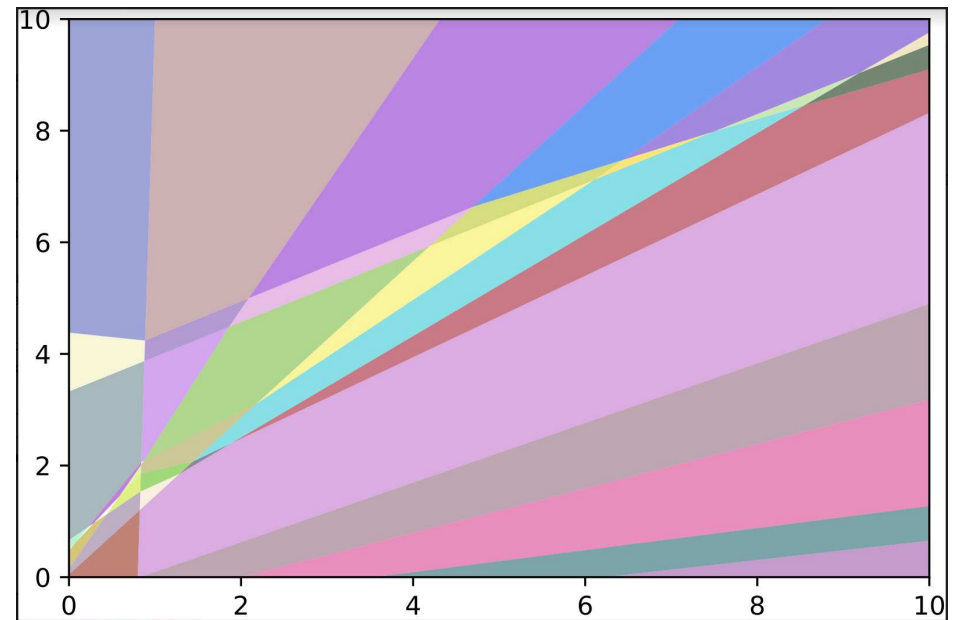
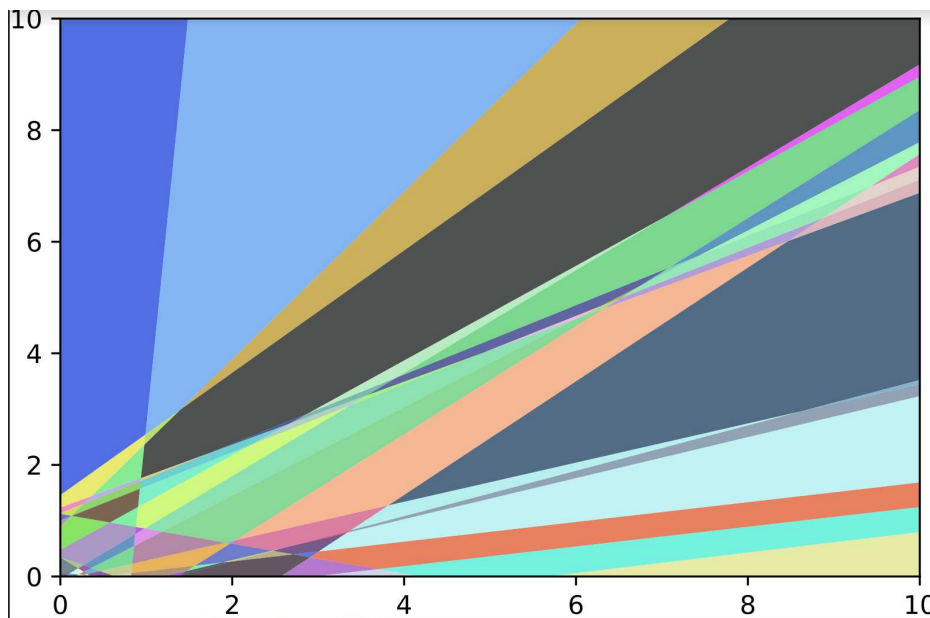
$$p(y \mid \mathbf{x}, \boldsymbol{\theta}) = \mathcal{N}(y \mid \mathbf{w}_{\mu}^t f(\mathbf{x}; \mathbf{w}_{\text{shared}}), \sigma_{+}(\mathbf{w}_{\sigma}^t f(\mathbf{x}; \mathbf{w}_{\text{shared}}))) \quad (16)$$

- Datos de media linealmente creciente en el tiempo, con oscilaciones estacionales, y varianza de crecimiento cuadrático



13.2.5. La importancia de la profundidad

- **Aproximador de funciones universal:** toda función (mínimamente suave) puede aproximarse tanto como se quiera mediante un **MLP de una capa oculta** con suficientes unidades
- Descomposición de $\mathbb{R}^2$ en un conjunto de regiones de decisión lineales producido por un MLP con activaciones ReLU y una capa de 25 unidades ocultas (izq.) o dos capas ocultas (dcha.):



- ▶ ***Redes profundas vs superficiales:*** las profundas, definidas de manera composicional o jerárquica, ofrecen mejores resultados
- ▷ Idea clave: las últimas capas aprovechan las características aprendidas en las primeras capas

13.2.6. La “revolución del aprendizaje profundo”

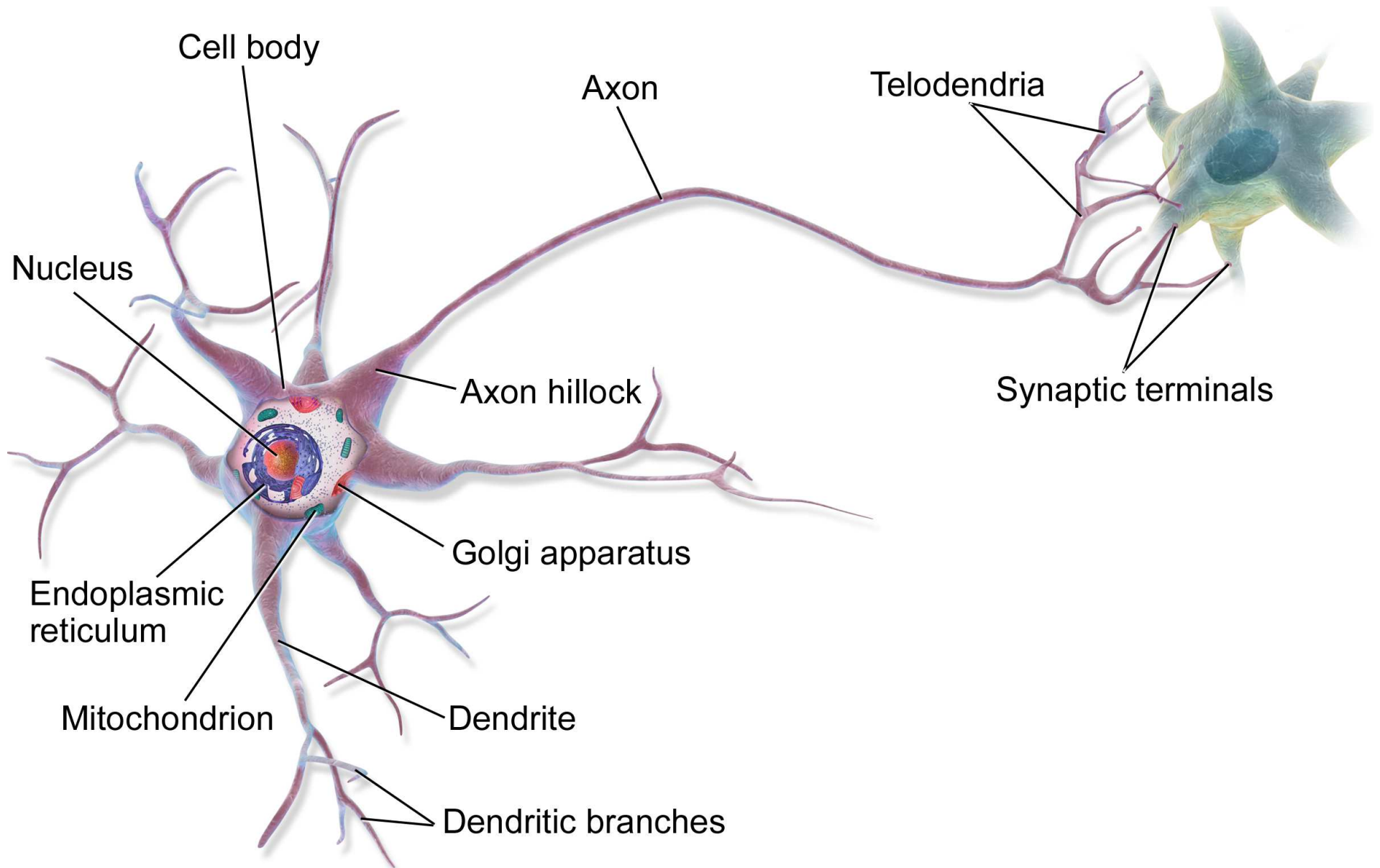
- ▶ **Gran popularidad a partir de los 2010:** aunque las ideas básicas tras las DNNs son conocidas durante varias décadas, su gran popularidad llega a partir de los 2010, desde el reconocimiento automático del habla (2011) y la clasificación de imágenes (2012)
- ▶ **GPUs y grandes conjuntos de datos supervisados:** son los principales factores que han contribuido al gran auge de las DNNs
- ▶ **Librerías de código abierto:** las grandes empresas tecnológicas han desarrollado librerías de código abierto que facilitan la aplicación de las DNNs a multitud de problemas
 - ▷ Tensorflow (Google) y PyTorch (Facebook) han sido especialmente populares

13.2.7. Conexiones con la biología

► *Modelo McCulloch-Pitts:*

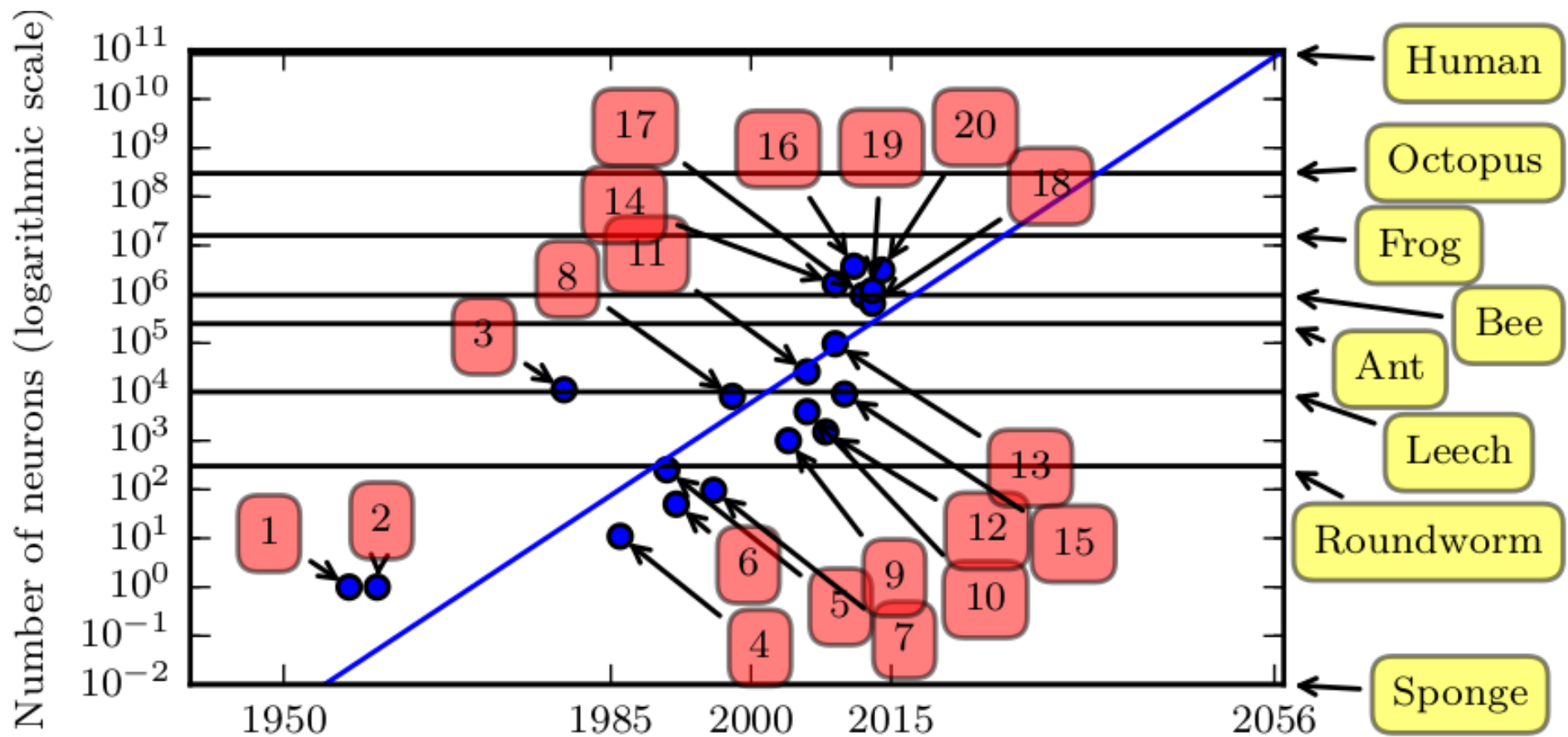
- ▷ Las neuronas tienen un comportamiento binario; sea $h_k \in \{0, 1\}$ el estado de activación o no de una neurona k
- ▷ h_k depende de las entradas de la neurona, $\mathbf{x} \in \mathbb{R}^D$
- ▷ h_k también depende de la fuerza de sus conexiones de entrada correspondientes, $\mathbf{w}_k \in \mathbb{R}^D$
- ▷ Si la actividad (total) de entrada, $a_k = \mathbf{w}_k^t \mathbf{x}$, supera cierto umbral de activación b_k , la neurona se activa

$$h_k(\mathbf{x}) = H(\mathbf{w}_k^t \mathbf{x} - b_k) \quad (17)$$



- ▶ ***Diferencias entre redes neuronales artificiales y biológicas:***
 - ▷ Las artificiales suelen modificar la fuerza de sus conexiones de entrada con retropropagación, cosa imposible en las biológicas pues no pueden enviar información hacia atrás por los axones
 - ▷ Las artificiales suelen ser hacia adelante, pero las biológicas tienen numerosas conexiones de retroalimentación
 - ▷ Las artificiales suelen usar neuronas simples, que aplican una función no-lineal sencilla a una suma ponderada, mientras que las biológicas tienen estructuras dendríticas complejas
 - ▷ Las artificiales son mucho más pequeñas que las biológicas
 - ▷ Las artificiales se diseñan para modelar una única tarea, mientras que las biológicas implementan múltiples tareas
- ▶ ***Las redes y “el problema de la IA”:*** no se sabe si modelos más realistas del cerebro pueden resolver mejor “el problema de la IA”

- ▶ Número de neuronas de las redes artificiales desde los años 1950 y comparación con el de algunos seres vivos:
 - ▷ Modelo 1: Perceptrón de Rosenblatt (1958)
 - ▷ Modelo 2: ADALINE de Widrow y Hoff (1960)
 - ▷ Modelo 3: Neocognitron de Fukushima (1980)
 - ▷ Modelo 4: MLP de Rumelhart, Hinton y Williams (1986)



13.3. Backpropagation

► *Algoritmo backpropagation (backprop):*

- ▷ Descubierta por Bryson y Ho (1969), re-descubierta por Werbos (1974); y popularizado por Rumelhart, Hinton y Williams (1986)
- ▷ Calcula el gradiente de una función de pérdida aplicada a la salida de una red con respecto a los parámetros de cada capa; se pasa a un algoritmo de optimización
- ▷ Puede verse como aplicaciones repetidas de la regla de la cadena si asumimos que el *grafo de computación* es una cadena lineal simple de capas apiladas

► *Diferenciación automática (autodiff):* versión general de backprop aplicable a grafos acíclicos arbitrarios

13.3.1. Diferenciación hacia adelante y hacia atrás

► Transformación f :

$$f : \mathbb{R}^n \rightarrow \mathbb{R}^m \quad o = f(x), \quad x \in \mathbb{R}^n, \quad o \in \mathbb{R}^m \quad (18)$$

► Asumimos que f es una composición de K funciones; por ejemplo, con $K = 4$, f_1 , f_2 , f_3 y f_4

$$f = f_4 \circ f_3 \circ f_2 \circ f_1 \quad o = f(x) \quad (19)$$

$$f_1 : \mathbb{R}^n \rightarrow \mathbb{R}^{m_1} \quad x_2 = f_1(x) \quad (20)$$

$$f_2 : \mathbb{R}^{m_1} \rightarrow \mathbb{R}^{m_2} \quad x_3 = f_2(x_2) \quad (21)$$

$$f_3 : \mathbb{R}^{m_2} \rightarrow \mathbb{R}^{m_3} \quad x_4 = f_3(x_3) \quad (22)$$

$$f_4 : \mathbb{R}^{m_3} \rightarrow \mathbb{R}^m \quad o = f_4(x_4) \quad (23)$$

- Jacobiana de f : producto de las Jacobianas de f_1 , f_2 , f_3 y f_4

$$\mathbf{J}_f(\mathbf{x}) = \frac{\partial \mathbf{o}}{\partial \mathbf{x}} \in \mathbb{R}^{m \times n} \quad (24)$$

$$\frac{\partial \mathbf{o}}{\partial \mathbf{x}} = \frac{\partial \mathbf{o}}{\partial \mathbf{x}_4} \frac{\partial \mathbf{x}_4}{\partial \mathbf{x}_3} \frac{\partial \mathbf{x}_3}{\partial \mathbf{x}_2} \frac{\partial \mathbf{x}_2}{\partial \mathbf{x}} \quad (25)$$

$$= \frac{\partial \mathbf{f}_4(\mathbf{x}_4)}{\partial \mathbf{x}_4} \frac{\partial \mathbf{f}_3(\mathbf{x}_3)}{\partial \mathbf{x}_3} \frac{\partial \mathbf{f}_2(\mathbf{x}_2)}{\partial \mathbf{x}_2} \frac{\partial \mathbf{f}_1(\mathbf{x})}{\partial \mathbf{x}} \quad (26)$$

$$= \mathbf{J}_{f_4}(\mathbf{x}_4) \mathbf{J}_{f_3}(\mathbf{x}_3) \mathbf{J}_{f_2}(\mathbf{x}_2) \mathbf{J}_{f_1}(\mathbf{x}) \quad (27)$$

- Recordatorio sobre la matriz Jacobiana, $\mathbf{J}_f(\mathbf{x}) \in \mathbb{R}^{m \times n}$:

$$\mathbf{J}_f(\mathbf{x}) = \frac{\partial \mathbf{f}(\mathbf{x})}{\partial \mathbf{x}} \quad (28)$$

$$= \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \cdots & \frac{\partial f_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_m}{\partial x_1} & \cdots & \frac{\partial f_m}{\partial x_n} \end{bmatrix} \quad (29)$$

$$= \begin{bmatrix} \nabla f_1(\mathbf{x})^t \\ \vdots \\ \nabla f_m(\mathbf{x})^t \end{bmatrix} \quad (30)$$

$$= \left[\frac{\partial \mathbf{f}}{\partial x_1}, \dots, \frac{\partial \mathbf{f}}{\partial x_n} \right] \quad (31)$$

- Caso $m = 1$: $\nabla \mathbf{f}(\mathbf{x}) = \mathbf{J}_f(\mathbf{x})^t$

► Diferenciación hacia adelante:

- Suponemos $n \leq m$ y queremos multiplicar $\mathbf{J}_f(\mathbf{x})$ por un vector columna $\mathbf{v}$ a la derecha

$$\mathbf{J}_f(\mathbf{x})\mathbf{v} = \underbrace{\mathbf{J}_{f_4}(\mathbf{x}_4)}_{m \times m_3} \underbrace{\mathbf{J}_{f_3}(\mathbf{x}_3)}_{m_3 \times m_2} \underbrace{\mathbf{J}_{f_2}(\mathbf{x}_2)}_{m_2 \times m_1} \underbrace{\mathbf{J}_{f_1}(\mathbf{x})}_{m_1 \times n} \underbrace{\mathbf{v}}_{n \times 1} \quad (32)$$

- Extracción de la columna j mediante producto Jacobiana-vector (JVP) con el vector base unitario $\mathbf{e}_j \in \mathbb{R}^n$, $j = 1, \dots, n$:

$$\mathbf{J}_f(\mathbf{x}) \mathbf{e}_j = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \dots & \frac{\partial f_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_m}{\partial x_1} & \dots & \frac{\partial f_m}{\partial x_n} \end{bmatrix} \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix} = \begin{bmatrix} \frac{\partial f_1}{\partial x_j} \\ \vdots \\ \frac{\partial f_m}{\partial x_j} \end{bmatrix} = \frac{\partial \mathbf{f}}{\partial x_j} \in \mathbb{R}^m \quad (33)$$

- $\mathbf{J}_f(\mathbf{x})$ puede calcularse por columnas con n JVPs

$$\begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \dots & \frac{\partial f_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_m}{\partial x_1} & \dots & \frac{\partial f_m}{\partial x_n} \end{bmatrix} = [\mathbf{J}_f(\mathbf{x}) \mathbf{e}_1, \dots, \mathbf{J}_f(\mathbf{x}) \mathbf{e}_n] \quad (34)$$

▷ **Algoritmo 5: forward mode differentiation**

1. $\mathbf{x}_1 := \mathbf{x}$

2. $\mathbf{v}_j := \mathbf{e}_j \in \mathbb{R}^n$ **para** $j = 1 : n$ // base canónica; obtiene $\mathbf{J}_f(\mathbf{x})$

3. **para** $k = 1 : K$:

4. $\mathbf{x}_{k+1} := \mathbf{f}_k(\mathbf{x}_k)$

5. $\mathbf{v}_j := \mathbf{J}_{\mathbf{f}_k}(\mathbf{x}_k) \mathbf{v}_j$ **para** $j = 1 : n$

6. **Retorna** $\mathbf{o} = \mathbf{x}_{K+1}$, $[\mathbf{J}_f(\mathbf{x})]_{:,j} = \mathbf{v}_j$ **para** $j = 1 : n$

→ Coste: $O(n^3)$ asumiendo $m = 1$ y $n = m_1 = m_2 = m_3$

► Diferenciación hacia atrás:

- Suponemos $n \geq m$ y queremos multiplicar $\mathbf{J}_f(\mathbf{x})$ por un vector fila $\mathbf{u}^t$ a la izquierda

$$\mathbf{u}^t \mathbf{J}_f(\mathbf{x}) = \underbrace{\mathbf{u}^t}_{1 \times m} \underbrace{\mathbf{J}_{f_4}(\mathbf{x}_4)}_{m \times m_3} \underbrace{\mathbf{J}_{f_3}(\mathbf{x}_3)}_{m_3 \times m_2} \underbrace{\mathbf{J}_{f_2}(\mathbf{x}_2)}_{m_2 \times m_1} \underbrace{\mathbf{J}_{f_1}(\mathbf{x})}_{m_1 \times n} \quad (35)$$

- Extracción de la fila i mediante producto vector-Jacobiana (VJP) con el vector base unitario $\mathbf{e}_i \in \mathbb{R}^m$, $i = 1, \dots, m$:

$$\mathbf{e}_i^t \mathbf{J}_f(\mathbf{x}) = [\mathbf{0}^t \mathbf{1} \mathbf{0}^t] \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \dots & \frac{\partial f_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_m}{\partial x_1} & \dots & \frac{\partial f_m}{\partial x_n} \end{bmatrix} = \begin{bmatrix} \frac{\partial f_i}{\partial x_1} & \dots & \frac{\partial f_i}{\partial x_n} \end{bmatrix} = \nabla f_i(\mathbf{x})^t \in \mathbb{R}^{1 \times n} \quad (36)$$

- $\mathbf{J}_f(\mathbf{x})$ puede calcularse por filas con m VJPs

$$\begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \dots & \frac{\partial f_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_m}{\partial x_1} & \dots & \frac{\partial f_m}{\partial x_n} \end{bmatrix} = \begin{bmatrix} \mathbf{e}_1^t \mathbf{J}_f(\mathbf{x}) \\ \vdots \\ \mathbf{e}_m^t \mathbf{J}_f(\mathbf{x}) \end{bmatrix} \quad (37)$$

▷ **Algoritmo 6: reverse mode differentiation**

1. $\mathbf{x}_1 := \mathbf{x}$

2. **para** $k = 1 : K$:

3. $\mathbf{x}_{k+1} := \mathbf{f}_k(\mathbf{x}_k)$

4. $\mathbf{u}_i := \mathbf{e}_i \in \mathbb{R}^m$ **para** $i = 1 : m$ // base canónica; obtiene $\mathbf{J}_f(\mathbf{x})$

5. **para** $k = K : 1$:

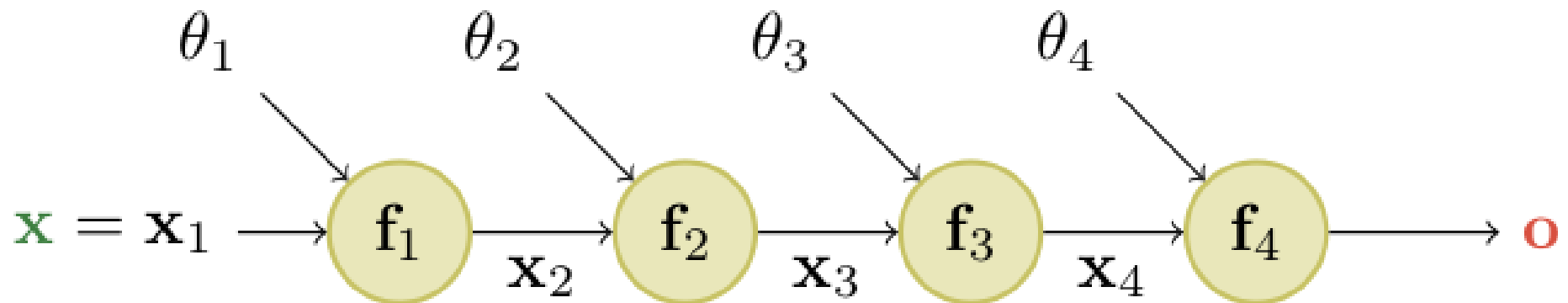
6. $\mathbf{u}_i^t := \mathbf{u}_i^t \mathbf{J}_{f_k}(\mathbf{x}_k)$ **para** $i = 1 : m$

7. **Retorna** $\mathbf{o} = \mathbf{x}_{K+1}$, $[\mathbf{J}_f(\mathbf{x})]_{i,:} = \mathbf{u}_i^t$ **para** $i = 1 : m$

→ Coste: $O(n^2)$ asumiendo $m = 1$ y $n = m_1 = m_2 = m_3$

13.3.2. Diferenciación hacia atrás para MLPs

- Asumimos que cada capa tiene su vector de parámetros, $\theta_1, \dots, \theta_4$:



- La transformación de interés se supone de salida escalar:

$$\mathcal{L} : \mathbb{R}^n \rightarrow \mathbb{R} \quad (38)$$

- ▷ *Ejemplo:* la pérdida ℓ_2 para un MLP de una capa oculta

$$\mathcal{L}((\mathbf{x}, \mathbf{y}), \boldsymbol{\theta}) = \frac{1}{2} \|\mathbf{y} - \mathbf{W}_2 \varphi(\mathbf{W}_1 \mathbf{x})\|_2^2 \quad (39)$$

► Modelo MLP: $f_k(x_k, \theta_k)$ denota la función en la capa k

$$\mathcal{L} = f_4 \circ f_3 \circ f_2 \circ f_1 \quad (40)$$

$$x_2 = f_1(x, \theta_1) = \mathbf{W}_1 x \quad (41)$$

$$x_3 = f_2(x_2, \theta) = \varphi(x_2) \quad (42)$$

$$x_4 = f_3(x_3, \theta_3) = \mathbf{W}_2 x_3 \quad (43)$$

$$\mathcal{L} = f_4(x_4, y) = \frac{1}{2} \|x_4 - y\|^2 \quad (44)$$

► Gradientes a hallar:

$$\frac{\partial \mathcal{L}}{\partial \theta_k} = (\nabla_{\theta_k} \mathcal{L})^t \in \mathbb{R}^{1 \times d_k} \quad \text{con } \theta_k \in \mathbb{R}^{d_k} \quad (45)$$

▷ El de la capa final se supone fácil de hallar: $\frac{\partial \mathcal{L}}{\partial x_4}$

- ▷ El resto se halla ordenadamente a partir del de la capa final

$$\frac{\partial \mathcal{L}}{\partial \mathbf{x}_4} \quad (46)$$

y las Jacobianas

$$\frac{\partial \mathbf{x}_k}{\partial \mathbf{x}_{k-1}} \in \mathbb{R}^{n_k \times n_{k-1}} \quad (47)$$

donde n_k es el número de unidades ocultas de la capa k :

$$\frac{\partial \mathcal{L}}{\partial \boldsymbol{\theta}_3} = \frac{\partial \mathcal{L}}{\partial \mathbf{x}_4} \frac{\partial \mathbf{x}_4}{\partial \boldsymbol{\theta}_3} \quad (48)$$

$$\frac{\partial \mathcal{L}}{\partial \boldsymbol{\theta}_2} = \frac{\partial \mathcal{L}}{\partial \mathbf{x}_4} \frac{\partial \mathbf{x}_4}{\partial \mathbf{x}_3} \frac{\partial \mathbf{x}_3}{\partial \boldsymbol{\theta}_2} \quad (49)$$

$$\frac{\partial \mathcal{L}}{\partial \boldsymbol{\theta}_1} = \frac{\partial \mathcal{L}}{\partial \mathbf{x}_4} \frac{\partial \mathbf{x}_4}{\partial \mathbf{x}_3} \frac{\partial \mathbf{x}_3}{\partial \mathbf{x}_2} \frac{\partial \mathbf{x}_2}{\partial \boldsymbol{\theta}_1} \quad (50)$$

► *Algoritmo 7: backprop para un MLP con K capas*

1. *// Paso forward*

2. $x_1 := x$

3. **para** $k = 1 : K$:

4. $x_{k+1} := f_k(x_k, \theta_k)$

5. *// Paso backward*

6. $u_{K+1} := 1$

7. **para** $k = K : 1$:

8. $g_k := u_{k+1}^t \frac{\partial f(x_k, \theta_k)}{\partial \theta_k}$

9. $u_k := u_{k+1}^t \frac{\partial f(x_k, \theta_k)}{\partial x_k}$

10. **Retorna** $\mathcal{L} = x_{K+1}, \quad \nabla_x \mathcal{L} = u_1, \quad \{\nabla_{\theta_k} \mathcal{L} = g_k : k = 1 : K\}$

13.3.3. Productos vector-Jacobiana para las capas

- Recordatorio sobre la matriz Jacobiana, $\mathbf{J}_f(\mathbf{x}) \in \mathbb{R}^{m \times n}$:

$$\mathbf{J}_f(\mathbf{x}) = \frac{\partial \mathbf{f}(\mathbf{x})}{\partial \mathbf{x}} = \begin{pmatrix} \frac{\partial f_1}{\partial x_1} & \dots & \frac{\partial f_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_m}{\partial x_1} & \dots & \frac{\partial f_m}{\partial x_n} \end{pmatrix} = \begin{pmatrix} \nabla f_1(\mathbf{x})^t \\ \vdots \\ \nabla f_m(\mathbf{x})^t \end{pmatrix} = \left(\frac{\partial \mathbf{f}}{\partial x_1}, \dots, \frac{\partial \mathbf{f}}{\partial x_n} \right) \quad (51)$$

donde $\nabla f_i(\mathbf{x})^t \in \mathbb{R}^{1 \times n}$ es la fila i y $\frac{\partial \mathbf{f}}{\partial x_j} \in \mathbb{R}^m$ la columna j

- backprop requiere hallar VJPs para:
 - ▷ Capa de entropía cruzada
 - ▷ Capa de no-linealidad elemental
 - ▷ Capa lineal

13.3.3.1. Capa de entropía cruzada

- Toma logits $\mathbf{x}$ y etiquetas $\mathbf{y}$; devuelve un escalar:

$$z = f(\mathbf{x}) \tag{52}$$

$$= \text{CrossEntropyWithLogits}(\mathbf{y}, \mathbf{x}) \tag{53}$$

$$= - \sum_c y_c \log(\mathcal{S}(\mathbf{x})_c) \tag{54}$$

$$= - \sum_c y_c \log p_c \tag{55}$$

donde $\mathbf{y}$ (one-hot) es la clase real y

$$p_c = \mathcal{S}(\mathbf{x})_c = \frac{\exp(x_c)}{\sum_{c'} \exp(x_{c'})} \tag{56}$$

la probabilidad predicha para la clase c

► La Jacobiana con respecto a $\mathbf{x}$ es:

$$\mathbf{J} = \frac{\partial z}{\partial \mathbf{x}} = (\mathbf{p} - \mathbf{y})^t \in \mathbb{R}^{1 \times C} \quad (57)$$

ya que, si la clase real es la c , $\mathbf{y} = \text{one-hot}(c)$, entonces

$$z = -\log(p_c) = -\log \left[\frac{\exp(x_c)}{\sum_j \exp(x_j)} \right] = \log \left[\sum_j \exp(x_j) \right] - x_c \quad (58)$$

por lo que, para todo $i = 1, \dots, C$,

$$\frac{\partial z}{\partial x_i} = \frac{\partial}{\partial x_i} \log \sum_j \exp(x_j) - \frac{\partial}{\partial x_i} x_c \quad (59)$$

$$= \frac{\exp(x_i)}{\sum_j \exp(x_j)} - \mathbb{1}(i = c) \quad (60)$$

$$= p_i - \mathbb{1}(i = c) \quad (61)$$

13.3.3.2. Capa de no-linealidad elemental

- Si $z = f(\mathbf{x}) = \varphi(\mathbf{x})$ es una no-linealidad elemental, $z_i = \varphi(x_i)$, su Jacobiana con respecto a la entrada es:

$$\mathbf{J} = \frac{\partial \mathbf{f}}{\partial \mathbf{x}} = \text{diag}(\varphi'(\mathbf{x})) \quad (62)$$

ya que, para toda fila i y columna j ,

$$\frac{\partial z_i}{\partial x_j} = \begin{cases} \varphi'(x_i) = \frac{d}{dx_i} \varphi(x_i) & \text{si } i = j \\ 0 & \text{otro caso} \end{cases} \quad (63)$$

- Si $\varphi(a) = \text{ReLU}(a) = \max(a, 0)$, su (sub)derivada es la función escalón de Heaviside,

$$\varphi'(a) = H(a) = \begin{cases} 0 & a \leq 0 \\ 1 & a > 0 \end{cases} \quad (64)$$

tomando $\varphi'(0) = 0$ (cualquier valor en $[0, 1]$ es subderivada en 0)

13.3.3.3. Capa lineal

► Dados $x \in \mathbb{R}^n$ y $W \in \mathbb{R}^{m \times n}$, $z \in \mathbb{R}^m$ es

$$z = f(x, W) = Wx \quad (65)$$

► La Jacobiana con respecto a x es:

$$J = \frac{\partial z}{\partial x} = W \quad (66)$$

ya que

$$z_i = \sum_{k=1}^n W_{ik} x_k \quad (67)$$

por lo que, para toda fila i y columna j ,

$$\frac{\partial z_i}{\partial x_j} = \frac{\partial}{\partial x_j} \sum_{k=1}^n W_{ik} x_k = \sum_{k=1}^n W_{ik} \frac{\partial x_k}{\partial x_j} \quad (68)$$

$$= \sum_{k=1}^n W_{ik} \mathbb{1}(k = j) = W_{ij} \quad (69)$$

- Jacobiana con respecto a $\mathbf{W}$, $\mathbf{J} = \frac{\partial \mathbf{z}}{\partial \mathbf{W}}$
 - ▷ Puede representarse como una matriz $m \times (m \times n)$, pero resulta complicada de manejar
 - ▷ Gradiente de z con respecto a un peso W_{ij} :

$$\frac{\partial z}{\partial W_{ij}} = (0 \ \cdots \ 0 \ x_j \ 0 \ \cdots \ 0)^t \quad (x_j \text{ en la posición } i) \quad (70)$$

ya que

$$z_k = \sum_{l=1}^m W_{kl} x_l \quad (71)$$

por lo que, para toda fila i y columna j ,

$$\frac{\partial z_k}{\partial W_{ij}} = \frac{\partial}{\partial W_{ij}} \sum_{l=1}^m x_l W_{kl} = \sum_{l=1}^m x_l \frac{\partial W_{kl}}{\partial W_{ij}} \quad (72)$$

$$= \sum_{l=1}^m x_l \mathbb{1}(i = k \text{ y } j = l) \quad (73)$$

► VJPs:

► De $\mathbf{u}^t \in \mathbb{R}^{1 \times m}$ y $\mathbf{J} \in \mathbb{R}^{m \times n}$

$$\mathbf{u}^t \frac{\partial \mathbf{z}}{\partial \mathbf{x}} = \mathbf{u}^t \mathbf{W} \in \mathbb{R}^{1 \times n} \quad (74)$$

► De $\mathbf{u}^t \in \mathbb{R}^{1 \times m}$ y $\frac{\partial \mathbf{z}}{\partial \mathbf{W}} \in \mathbb{R}^{m \times (m \times n)}$

$$\left[\mathbf{u}^t \frac{\partial \mathbf{z}}{\partial \mathbf{W}} \right]_{1,:} = \mathbf{u} \mathbf{x}^t \in \mathbb{R}^{m \times n} \quad (75)$$

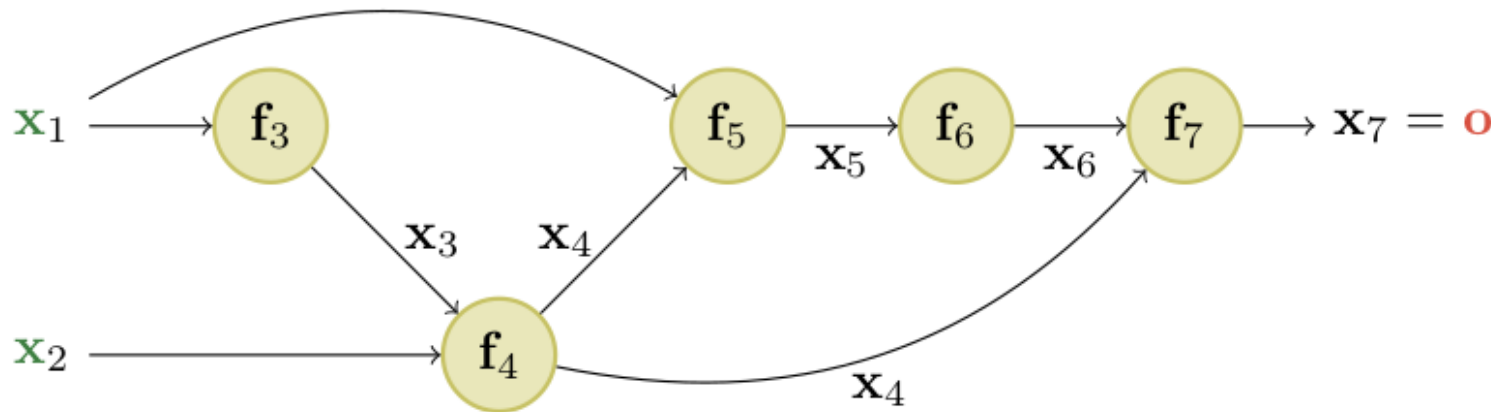
pues

$$\mathbf{u}^t \frac{\partial \mathbf{z}}{\partial W_{ij}} = \sum_{k=1}^m u_k \frac{\partial z_k}{\partial W_{ij}} = u_i x_j \quad (76)$$

13.3.4. Grafos de computación

- **Grafo de computación:** sinónimo de DNN moderna, esto es, una composición de funciones diferenciables que transforma una entrada en una salida con un **grafo dirigido acíclico (DAG)**

- **Ejemplo:** $f(x_1, x_2) = x_2 \exp(x_1) \sqrt{x_1 + x_2 \exp(x_1)}$ (77)



$$x_3 = f_3(x_1) = \exp(x_1) \quad (78)$$

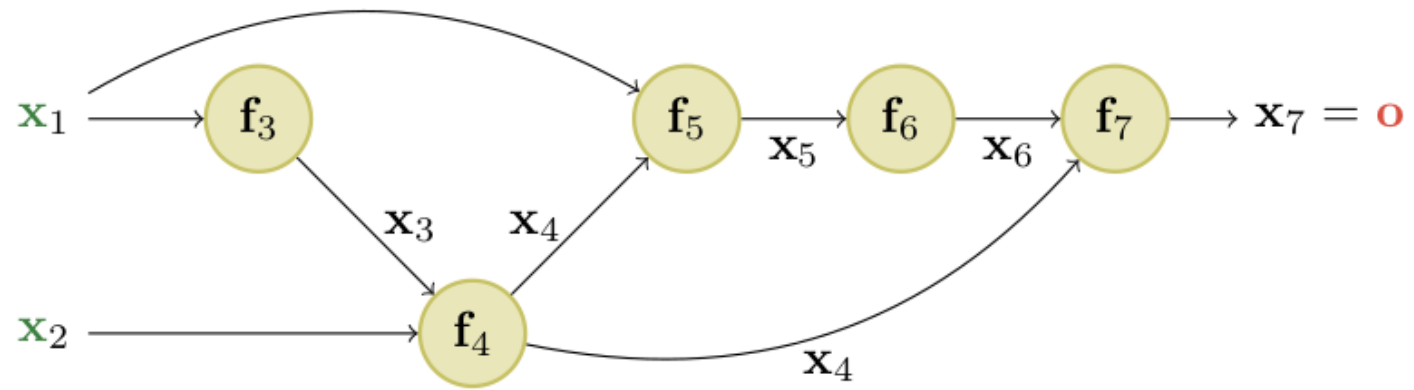
$$x_4 = f_4(x_2, x_3) = x_2 x_3 \quad (79)$$

$$x_5 = f_5(x_1, x_4) = x_1 + x_4 \quad (80)$$

$$x_6 = f_6(x_5) = \sqrt{x_5} \quad (81)$$

$$x_7 = f_7(x_4, x_6) = x_4 x_6 \quad (82)$$

► Ej. (cont.):



- Nodos numerados en orden topológico (padres antes que hijos)
- El paso backward suma gradientes para caminos múltiples:

$$\frac{\partial o}{\partial x_4} = \frac{\partial o}{\partial x_5} \frac{\partial x_5}{\partial x_4} + \frac{\partial o}{\partial x_7} \frac{\partial x_7}{\partial x_4} \quad (83)$$

- Cálculos en orden topológico inverso evitan cálculos repetidos:

$$\frac{\partial o}{\partial x_7} = \frac{\partial x_7}{\partial x_7} = \mathbf{I}_m \quad (84)$$

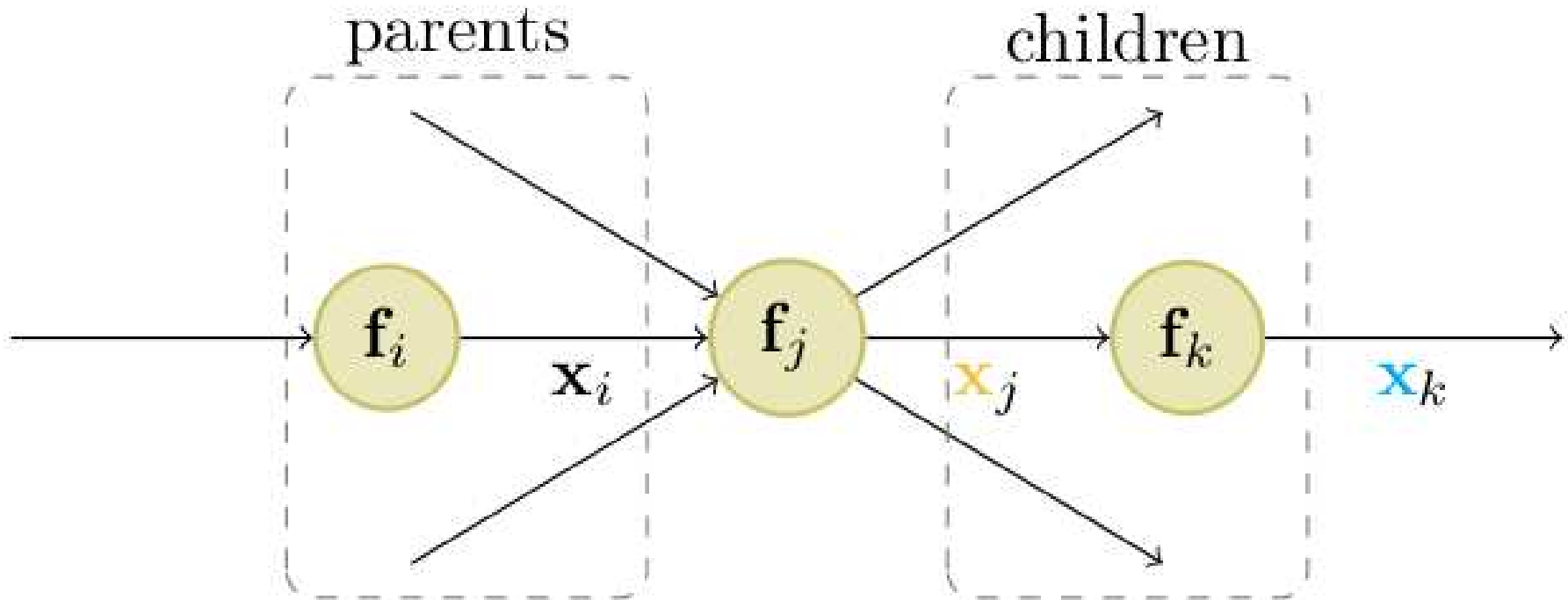
$$\frac{\partial o}{\partial x_6} = \frac{\partial o}{\partial x_7} \frac{\partial x_7}{\partial x_6} \quad (85)$$

$$\frac{\partial o}{\partial x_5} = \frac{\partial o}{\partial x_6} \frac{\partial x_6}{\partial x_5} \quad (86)$$

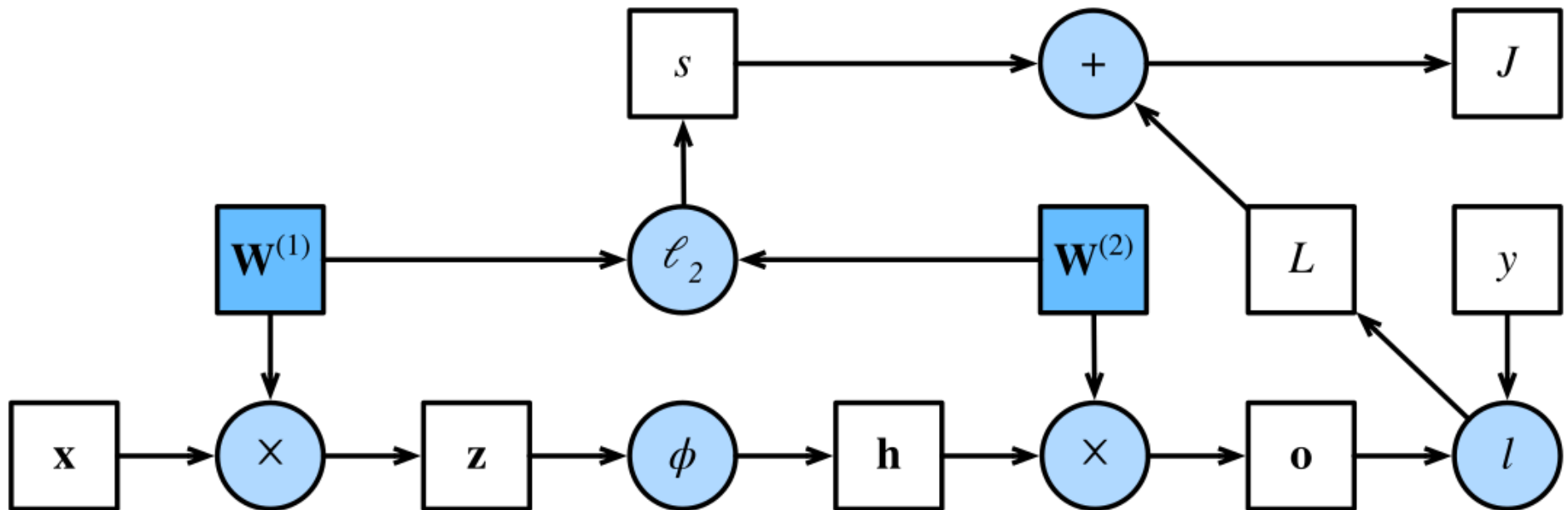
$$\frac{\partial o}{\partial x_4} = \frac{\partial o}{\partial x_5} \frac{\partial x_5}{\partial x_4} + \frac{\partial o}{\partial x_7} \frac{\partial x_7}{\partial x_4} \quad (87)$$

- En general, el gradiente para un nodo j se calcula como la suma, para todo hijo k , de su gradiente (ya calculado) o *adjunto*, $\frac{\partial o}{\partial x_k}$, multiplicado por la Jacobiana $\frac{\partial x_k}{\partial x_j}$:

$$\frac{\partial o}{\partial x_j} = \sum_{k \in \text{children}(j)} \frac{\partial o}{\partial x_k} \frac{\partial x_k}{\partial x_j} \quad (88)$$



- **Grafos estáticos y dinámicos:** son estáticos si se calculan en preproceso; dinámicos si se calculan *“just in time”*, trazando la ejecución de una función para una entrada dada.
- *Ejemplo:* grafo de MLP con una capa oculta y penalización de pesos que calcula pre-activaciones lineales $z = \mathbf{W}^{(1)}\mathbf{x}$, activaciones ocultas $\mathbf{h} = \phi(\mathbf{z})$, salidas lineales $\mathbf{o} = \mathbf{w}^{(2)}\mathbf{h}$, pérdida $L = \ell(\mathbf{o}, \mathbf{y})$, regularizador $s = \frac{\lambda}{2}(\|\mathbf{W}^{(1)}\|_F^2 + \|\mathbf{W}^{(2)}\|_F^2)$ y pérdida total $J = L + s$:



13.4. Entrenamiento de redes neuronales

- ▶ Aproximación convencional: minimización de la NLL

$$\mathcal{L}(\boldsymbol{\theta}) = -\log p(\mathcal{D} \mid \boldsymbol{\theta}) = -\sum_{n=1}^N \log p(\mathbf{y}_n \mid \mathbf{x}_n; \boldsymbol{\theta}) \quad (89)$$

- ▶ Es usual añadir un regularizador como la log-prior negativa
- ▶ Calculamos el gradiente de la pérdida con backprop y se lo pasamos a un optimizador estándar; Adam es muy popular
- ▶ En la práctica se suelen presentar algunos problemas para lo cuales se tienen soluciones más o menos populares

13.4.1. Ajuste del factor de aprendizaje

- ▶ Necesario para garantizar la convergencia a una buena solución

13.4.2. Gradientes que desaparecen y explotan

- ▶ *Vanishing and exploding gradients:* durante el entrenamiento de modelos muy profundos, el gradiente tiende a hacerse
 - ▷ muy pequeño (desaparecer)
 - ▷ o muy grande (explotar)

► *Radio espectral de la Jacobiana, λ :*

- Gradiente de la pérdida con respecto a un nodo de la capa l :

$$\frac{\partial \mathcal{L}}{\partial \mathbf{z}_l} = \frac{\partial \mathcal{L}}{\partial \mathbf{z}_{l+1}} \frac{\partial \mathbf{z}_{l+1}}{\partial \mathbf{z}_l} = \mathbf{J}_l \mathbf{g}_{l+1} \quad (90)$$

donde $\mathbf{J}_l = \frac{\partial \mathbf{z}_{l+1}}{\partial \mathbf{z}_l}$ es la Jacobiana y $\mathbf{g}_{l+1} = \frac{\partial \mathcal{L}}{\partial \mathbf{z}_{l+1}}$ el gradiente en $l+1$

- Si $\mathbf{J}_l$ es constante en las capas, la contribución del gradiente de la última capa, $\mathbf{g}_L$, a la capa l es $\mathbf{J}^{L-l} \mathbf{g}_L$, por lo que el comportamiento del sistema depende de los vectores propios de $\mathbf{J}$
- Sea λ el máximo de los valores propios de $\mathbf{J}$ en valor absoluto
 - ⇒ si $\lambda > 1$, el gradiente puede explotar;
 - ⇒ si $\lambda < 1$, puede desaparecer

- ▶ **Gradient clipping:** el problema de los gradientes que explotan se suaviza recortando la magnitud del gradiente hasta un c dado:

$$\mathbf{g}' = \min \left(1, \frac{c}{\|\mathbf{g}\|} \right) \mathbf{g} \quad (91)$$

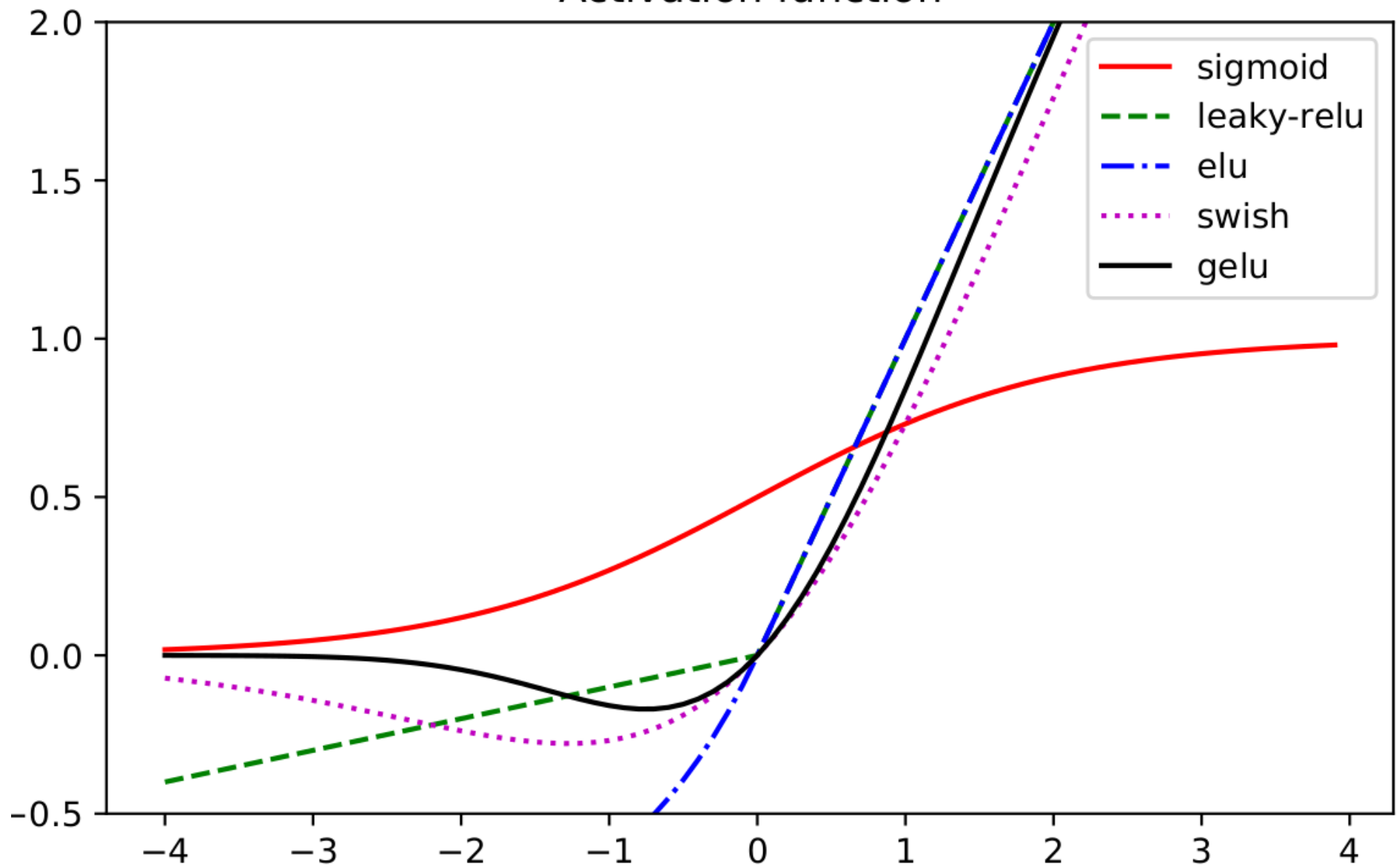
- ▶ Soluciones para el problema de los gradientes que desaparecen:
 - ▷ Modificar las funciones de activación para evitar que el gradiente sea demasiado grande o pequeño
 - ▷ Modificar la arquitectura para que las actualizaciones sean aditivas en lugar de multiplicativas
 - ▷ Modificar la arquitectura para estandarizar las activaciones de cada capa, de manera que la distribución de las activaciones sobre los datos se mantenga constante durante el entrenamiento
 - ▷ Escoger los valores iniciales de los parámetros cuidadosamente

13.4.3. Funciones de activación no saturantes

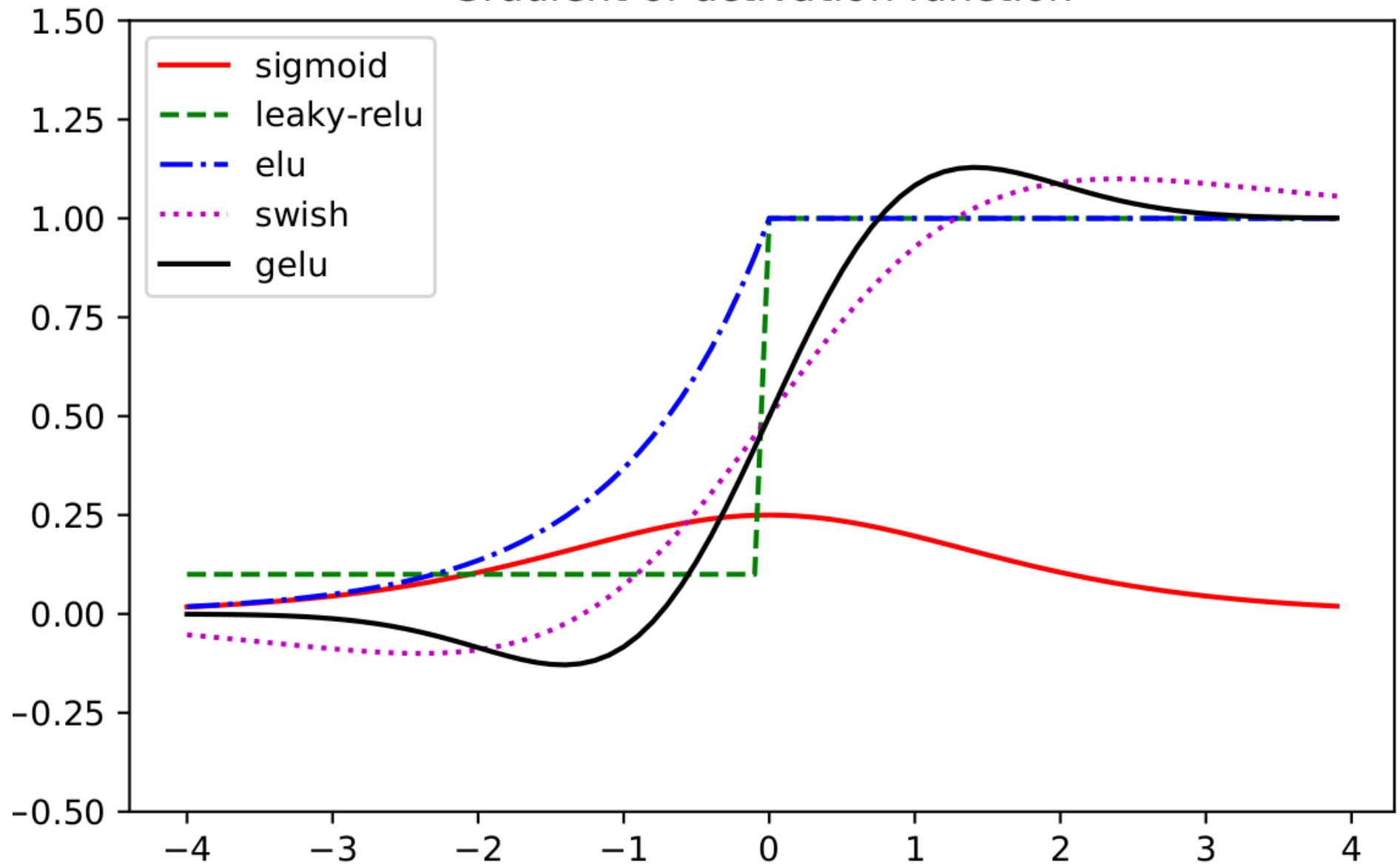
► Funciones de activación populares:

Nombre	Definición	Rango
Sigmoide	$\sigma(a) = \frac{1}{1 + e^{-a}}$	$[0, 1]$
Tangente hiperbólica	$\tanh(a) = 2\sigma(2a) - 1$	$[-1, 1]$
Softplus	$\sigma_+(a) = \log(1 + e^a)$	$[0, \infty]$
Rectified linear unit	$\text{ReLU}(a) = \max(a, 0)$	$[0, \infty]$
Leaky ReLU	$\max(a, 0) + \alpha \min(a, 0)$	$[-\infty, \infty]$
Exponential linear unit	$\max(a, 0) + \min(\alpha(e^a - 1), 0)$	$[-\infty, \infty]$
Swish	$a\sigma(a)$	$[-\infty, \infty]$
GELU	$a\Phi(a)$	$[-\infty, \infty]$

Activation function



Gradient of activation function



- ▶ La sigmoide en régimen de saturación favorece gradientes nulos:
 - ▷ Consideremos una capa que calcula $z = \sigma(\mathbf{W}x)$ con:

$$\varphi(a) = \sigma(a) = \frac{1}{1 + \exp(-a)} \quad (92)$$

Si los pesos se inicializan con valores grandes (en términos absolutos), será fácil que $a = \mathbf{W}x$ tome valores también grandes, por lo que z saturará cerca de 0 o 1 a causa de la sigmoide

- ▷ La derivada de la sigmoide es:

$$\varphi'(a) = \sigma(a)(1 - \sigma(a)) \quad (93)$$

- ▷ El gradiente de la pérdida con respecto a x y $\mathbf{W}$ es:

$$\frac{\partial \mathcal{L}}{\partial x} = \mathbf{W}^t \delta = \mathbf{W}^t z(1 - z) \quad (94)$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}} = \delta x^t = z(1 - z)x^t \quad (95)$$

- ▷ Luego, si z está cerca de 0 o 1, el gradiente se aproximará a 0

13.4.3.1. ReLU

- **Rectified linear unit (ReLU):** “apaga” entradas negativas

$$\text{ReLU}(a) = \max(a, 0) = a \mathbb{1}(a > 0) \quad (96)$$

- Gradiente:

$$\text{ReLU}'(a) = \mathbb{1}(a > 0) \quad (97)$$

- Si $z = \text{ReLU}(\mathbf{W}x)$, los gradientes con respecto a $\mathbf{W}$ y x no desaparecen siempre y cuando z sea positiva:

$$\frac{\partial \mathcal{L}}{\partial x} = \mathbf{W}^t \mathbb{1}(z > 0) \quad (98)$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}} = \mathbb{1}(z > 0) x^t \quad (99)$$

- **Problema de la “ReLU muerta”:** si los pesos se inicializan con valores negativos grandes, será fácil que algunas componentes de $a = \mathbf{W}x$ también tomen valores negativos grandes, z saturará cerca de 0 y el gradiente con respecto a $\mathbf{W}$ tenderá a 0

13.4.3.2. ReLU no saturante

- **Leaky ReLU:** dado $0 < \alpha < 1$, se define como:

$$\text{LReLU}(a; \alpha) = \max(\alpha a, a) \quad (100)$$

Con pendiente 1 para entradas positivas y α para negativas, garantiza señal a las capas previas incluso con entrada negativa

- **ReLU paramétrica:** LReLU con α aprendido en lugar de prefijado
- **ELU:** más suave que LReLU

$$\text{ELU}(a; \alpha) = \begin{cases} \alpha(e^a - 1) & \text{si } a \leq 0 \\ a & \text{si } a > 0 \end{cases} \quad (101)$$

- **Self-normalizing ELU (SELU):**

$$\text{SELU}(a; \alpha, \lambda) = \lambda \text{ELU}(a; \alpha) \quad (102)$$

Con α y λ cuidadosamente ajustados, garantiza que la salida de cada capa se halla estandarizada (si la entrada también lo está)

13.4.3.3. Otras opciones

- **swish**: hallada mediante optimización caja-negra e independientemente propuesta como **Sigmoid Linear Unit (SiLU)**

$$\text{swish}(a; \beta) = a\sigma(\beta a) \quad (103)$$

- **Gaussian Error Linear Unit (GELU)**:

$$\text{GELU}(a) = a\Phi(a) \quad (104)$$

donde $\Phi(a)$ es la cdf de una normal estándar:

$$\Phi(a) = \Pr(\mathcal{N}(0, 1) \leq a) = \frac{1}{2} \left(1 + \text{erf} \left(\frac{a}{\sqrt{2}} \right) \right) \quad (105)$$

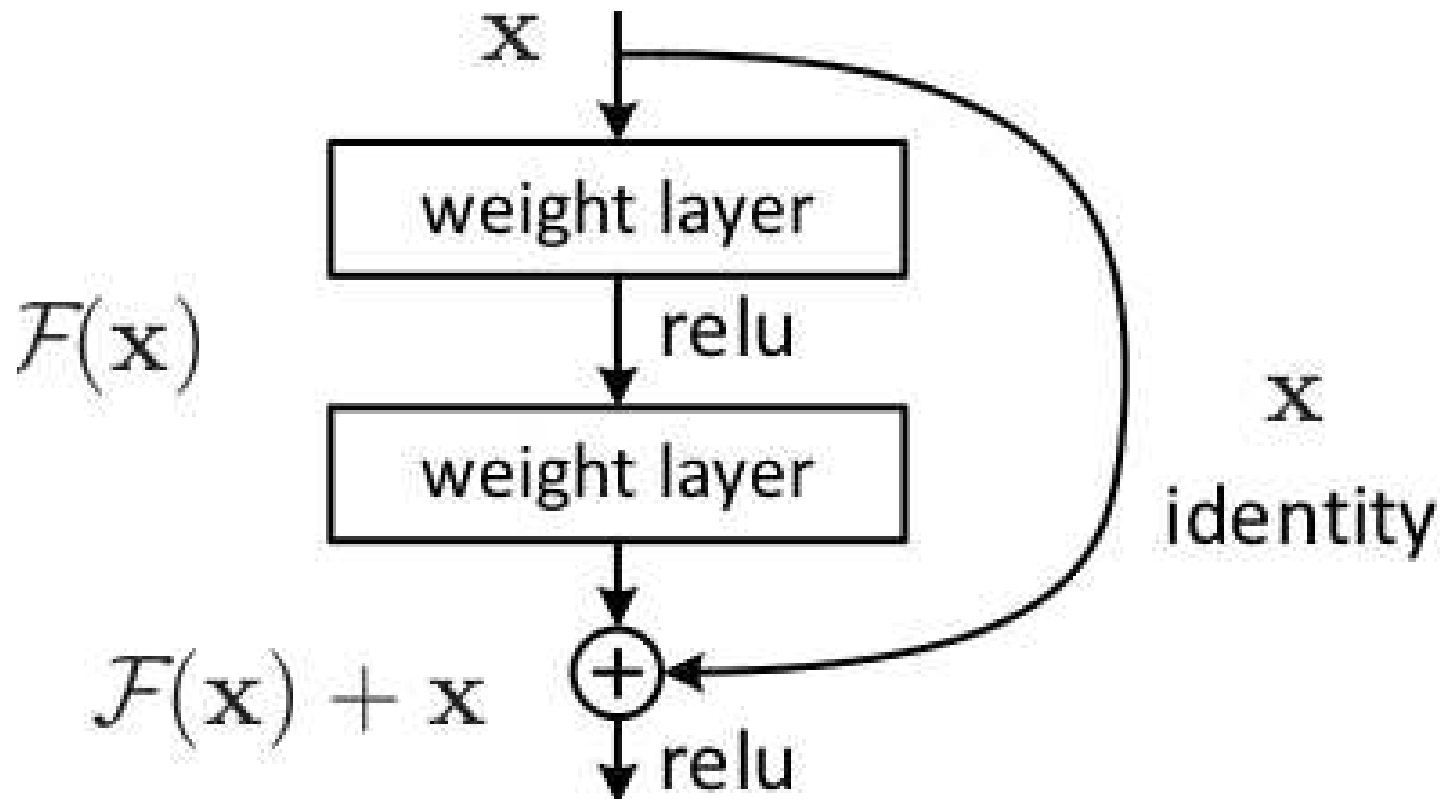
- ▷ A diferencia de la mayoría, no es convexa ni monótona
- ▷ Versión suave de la ReLU que sustituye $\mathbb{1}(a > 0)$ por $\Phi(a)$
- ▷ Versión adaptativa de dropout que multiplica la entrada por una máscara binaria escalar $m \sim \text{Ber}(\Phi(a))$, donde la posibilidad de omisión es $1 - \Phi(a)$, por lo que la salida esperada es $a\Phi(a)$
- ▷ Aproximación swish: $\text{GELU}(a) \approx \text{swish}(a; \beta)$ con $\beta = 1.702$

13.4.4. Conexiones residuales

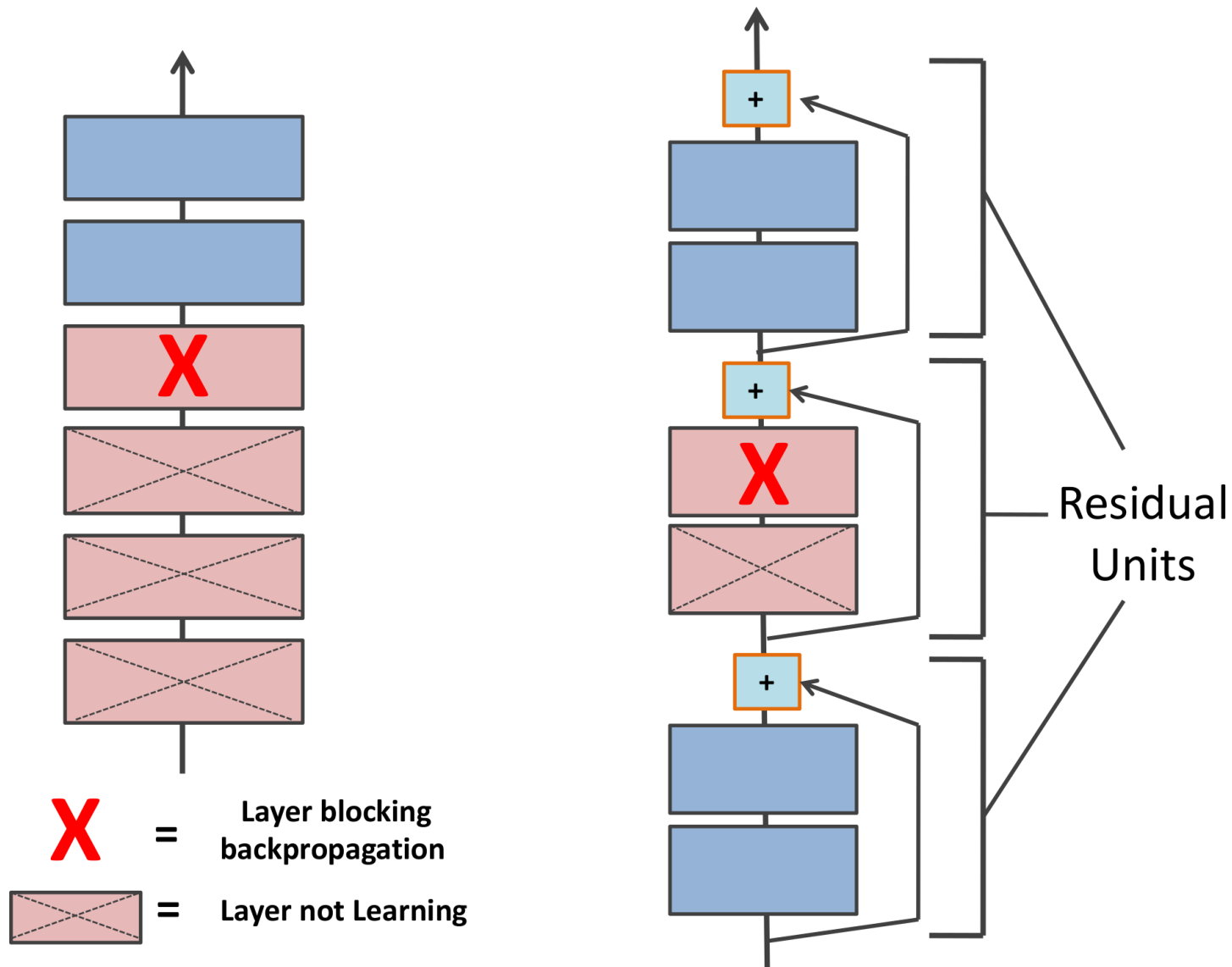
► *Residual network (ResNet)*: cada capa es un *bloque residual*

$$\mathcal{F}'_l = \mathcal{F}_l(x) + x \quad (106)$$

donde $\mathcal{F}_l$ es una transformación no lineal estándar (lineal - activación - lineal) que halla el término residual o delta a añadir a la entrada (perturbada) para generar la salida deseada



- Un modelo con conexiones residuales tiene el mismo número de parámetros que sin ellas, pero es más fácil de entrenar ya que los gradientes fluyen directamente de la salida a las primeras capas



- Activaciones de salida en función de cualquier capa previa l :

$$\mathbf{z}_L = \mathbf{z}_l + \sum_{i=l}^{L-1} \mathcal{F}_i(\mathbf{z}_i; \boldsymbol{\theta}_i) \quad (107)$$

- Gradiente de $\mathcal{L}$ con respecto a los parámetros de la capa l :

$$\frac{\partial \mathcal{L}}{\partial \boldsymbol{\theta}_l} = \frac{\partial \mathbf{z}_l}{\partial \boldsymbol{\theta}_l} \frac{\partial \mathcal{L}}{\partial \mathbf{z}_l} \quad (108)$$

$$= \frac{\partial \mathbf{z}_l}{\partial \boldsymbol{\theta}_l} \frac{\partial \mathcal{L}}{\partial \mathbf{z}_L} \frac{\partial \mathbf{z}_L}{\partial \mathbf{z}_l} \quad (109)$$

$$= \frac{\partial \mathbf{z}_l}{\partial \boldsymbol{\theta}_l} \frac{\partial \mathcal{L}}{\partial \mathbf{z}_L} \left(1 + \sum_{i=l}^{L-1} \frac{\partial \mathcal{F}_i(\mathbf{z}_i; \boldsymbol{\theta}_i)}{\partial \mathbf{z}_l} \right) \quad (110)$$

$$= \frac{\partial \mathbf{z}_l}{\partial \boldsymbol{\theta}_l} \frac{\partial \mathcal{L}}{\partial \mathbf{z}_L} + \dots \quad (111)$$

Luego el gradiente en la capa l depende directamente del de la capa L con independencia de la profundidad de la red

13.4.5. Inicialización de parámetros

13.4.5.1. Esquemas de inicialización heurísticos

► **Inicialización Xavier** o **Glorot**: dada una unidad lineal de **fan-in** n_{in} y **fan-out** n_{out} , con $o_i = \sum_{j=1}^{n_{\text{in}}} w_{ij} x_j$, muestreamos al azar:

$$w_{ij} \sim \mathcal{N}(0, \sigma^2) \quad \text{con} \quad \sigma^2 = \frac{2}{n_{\text{in}} + n_{\text{out}}} \quad (112)$$

► Si $w_{ij} \sim \mathcal{N}(0, \sigma^2)$, $\mathbb{E}[x_j] = 0$ y $\mathbb{V}[x_j] = \gamma^2$, con x_j y w_{ij} indep.:

$$\mathbb{E}[o_i] = \sum_{j=1}^{n_{\text{in}}} \mathbb{E}[w_{ij} x_j] = \sum_{j=1}^{n_{\text{in}}} \mathbb{E}[w_{ij}] \mathbb{E}[x_j] = 0 \quad (113)$$

$$\mathbb{V}[o_i] = \mathbb{E}[o_i^2] - \mathbb{E}[o_i]^2 = \sum_{j=1}^{n_{\text{in}}} \mathbb{E}[w_{ij}^2 x_j^2] = \sum_{j=1}^{n_{\text{in}}} \mathbb{E}[w_{ij}^2] \mathbb{E}[x_j^2] = n_{\text{in}} \sigma^2 \gamma^2 \quad (114)$$

por lo que $n_{\text{in}} \sigma^2 = 1$ evita que la varianza explote

► Análogamente, $n_{\text{out}} \sigma^2 = 1$ evita que la varianza de los gradientes explote en el paso backward

► La σ^2 escogida pretende garantizar ambas condiciones

► **Inicialización LeCun:** $\sigma^2 = \frac{1}{n_{\text{in}}}$, esto es, Glorot con $n_{\text{in}} = n_{\text{out}}$

► **Inicialización He:** $\sigma^2 = \frac{2}{n_{\text{in}}}$

► **Inicialización con una uniforme:** si $w_{ij} \sim \text{Unif}(-a, a)$

$$\mathbb{E}[o_i] = 0 \quad (115)$$

$$\mathbb{V}[o_i] = \frac{a^2}{3} \quad (116)$$

por lo que debemos fijar

$$a = \sqrt{\frac{6}{n_{\text{in}} + n_{\text{out}}}} \quad (117)$$

► **Unidades no lineales:** depende la función de activación

▷ lineal, tanh, logística y softmax: Glorot

▷ ReLU i variantes: He

▷ SELU: LeCun

13.4.5.2. Inicializaciones dirigidas por los datos

► *Inicialización layer-sequential unit-variance (LSUV):*

- ▷ Inicializamos los pesos de cada capa con matrices ortonormales; p.e., muestreando $w \in \mathcal{N}(0, \mathbf{I})$, reformateando w como una matriz $\mathbf{W}$ y hallando una base ortonormal con QR o SVD
- ▷ Para cada capa l , calculamos la varianza de las activaciones en un minibatch, v_l , y re-escalamos $\mathbf{W}_l = \mathbf{W}_l / \sqrt{v_l}$

13.4.5.3. Interpretación como espacio funcional

- MLP con L capas ocultas y salida lineal:

$$f(\mathbf{x}; \boldsymbol{\theta}) = \mathbf{W}_L(\cdots \varphi(\mathbf{W}_2 \varphi(\mathbf{W}_1 \mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2) \cdots) + \mathbf{b}_L \quad (118)$$

- Prior Gaussiano factorizado para los parámetros:

$$\mathbf{W}_\ell \sim \mathcal{N}(\mathbf{0}, \alpha_\ell^2 \mathbf{I}), \quad \mathbf{b}_\ell \sim \mathcal{N}(\mathbf{0}, \beta_\ell^2 \mathbf{I}) \quad (119)$$

- Re-parametrización:

$$\mathbf{W}_\ell = \alpha_\ell \boldsymbol{\eta}_\ell, \quad \boldsymbol{\eta}_\ell \sim \mathcal{N}(\mathbf{0}, \mathbf{I}), \quad \mathbf{b}_\ell = \beta_\ell \boldsymbol{\epsilon}_\ell, \quad \boldsymbol{\epsilon}_\ell \sim \mathcal{N}(\mathbf{0}, \mathbf{I}) \quad (120)$$

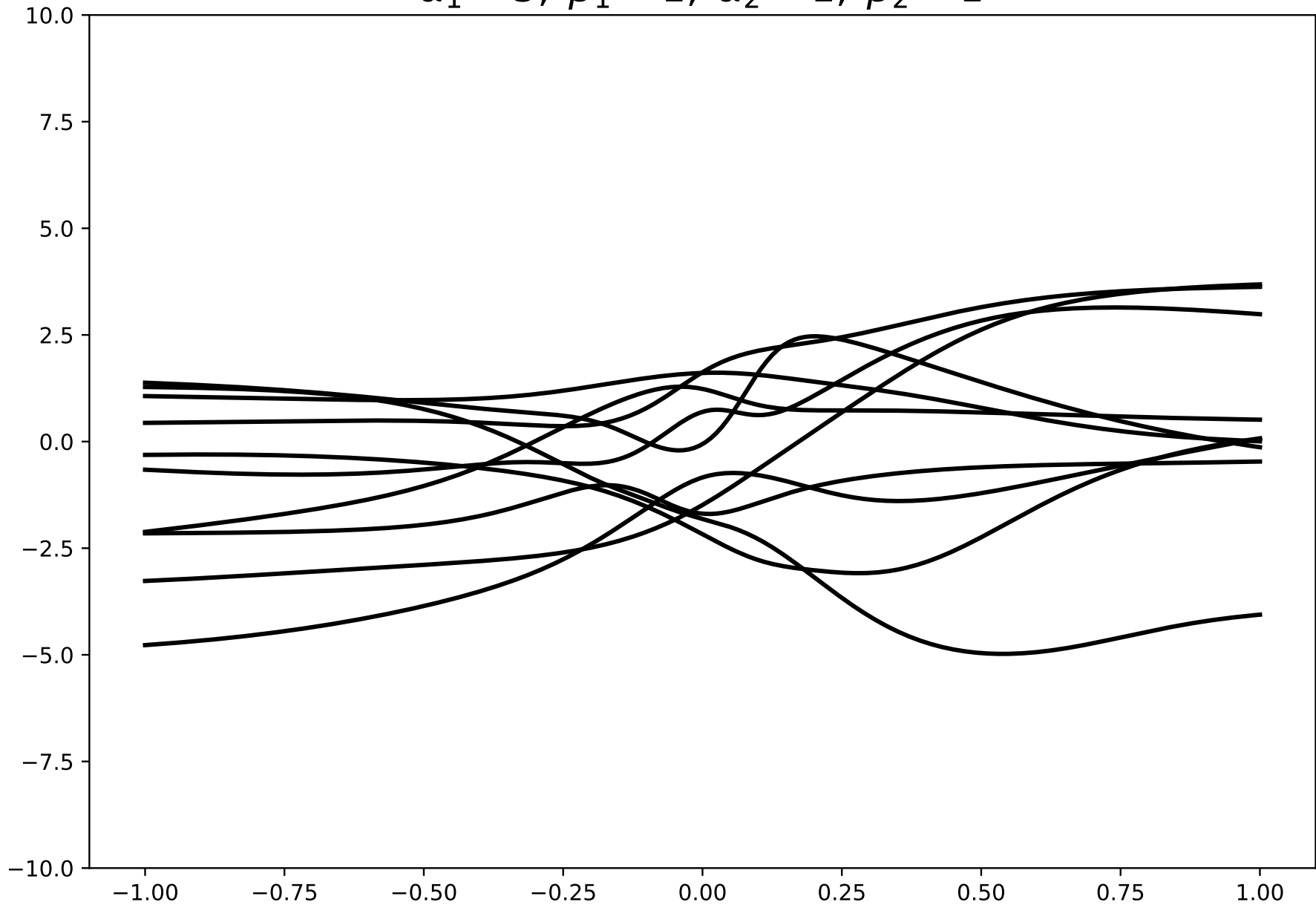
- Función aleatoria en términos de los hiperparámetros del prior:

$$f(\mathbf{x}; \boldsymbol{\alpha}, \boldsymbol{\beta}) = \alpha_L \boldsymbol{\eta}_L(\cdots \varphi(\alpha_1 \boldsymbol{\eta}_1 \mathbf{x} + \beta_1 \boldsymbol{\epsilon}_1) \cdots) + \beta_L \boldsymbol{\epsilon}_L \quad (121)$$

- *Ejemplos:* $f : \mathbb{R} \rightarrow \mathbb{R}$, $L = 2$, $\varphi(a) = \boldsymbol{\sigma}(a)$

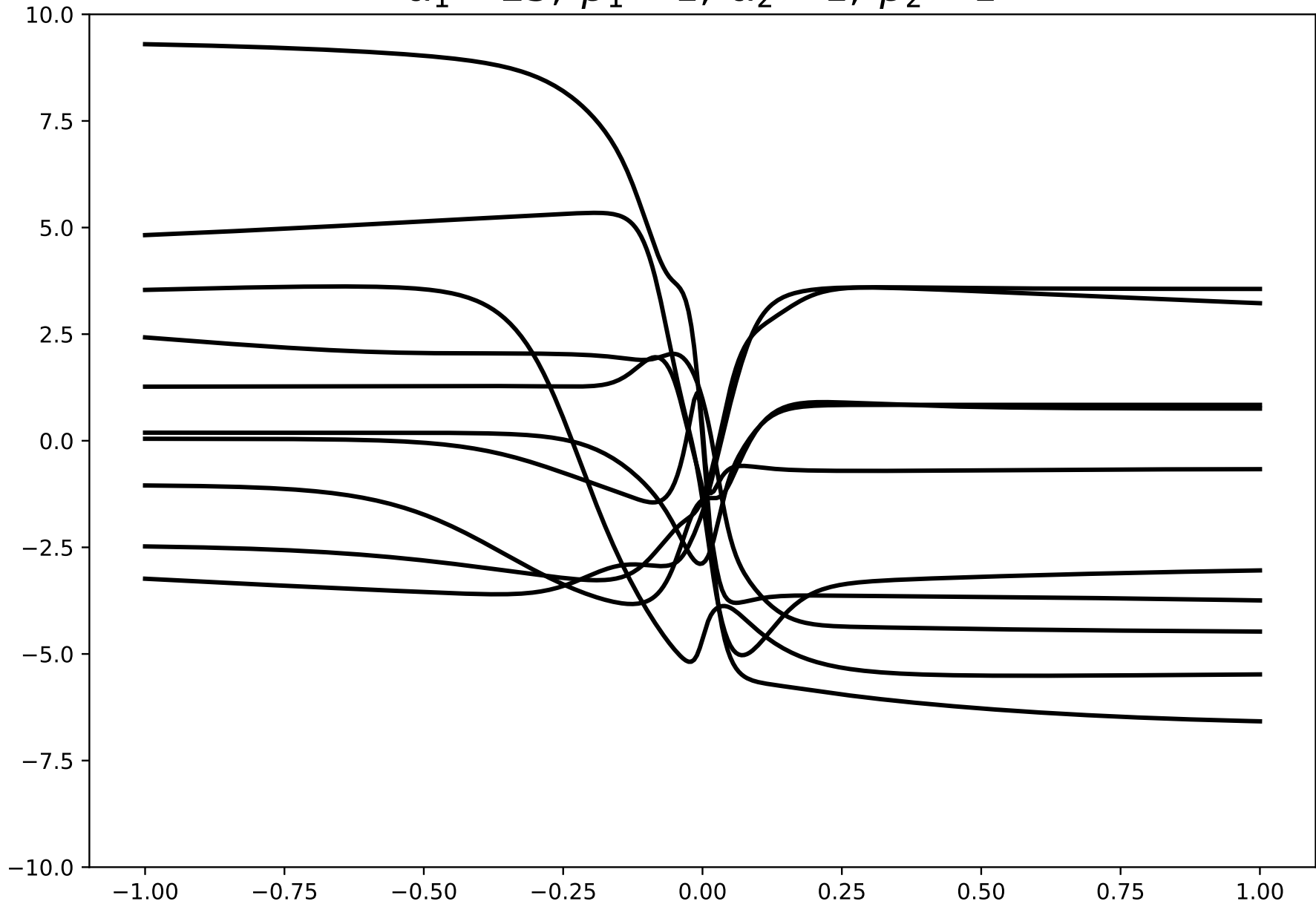
► Funciones generadas con hiperparámetros de base:

$$\alpha_1 = 5, \beta_1 = 1, \alpha_2 = 1, \beta_2 = 1$$



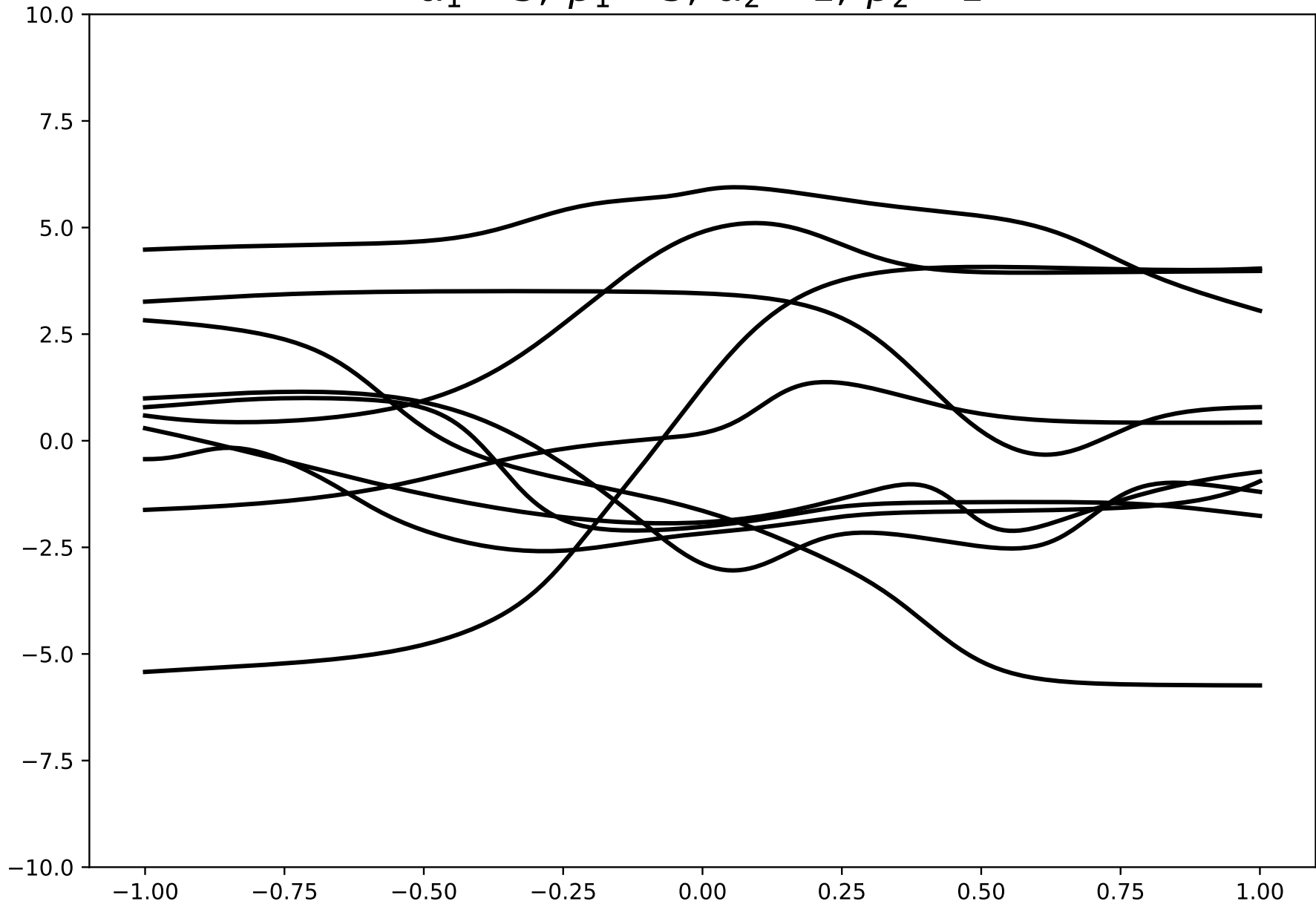
► Funciones generadas incrementando α_1 :

$$\alpha_1 = 25, \beta_1 = 1, \alpha_2 = 1, \beta_2 = 1$$



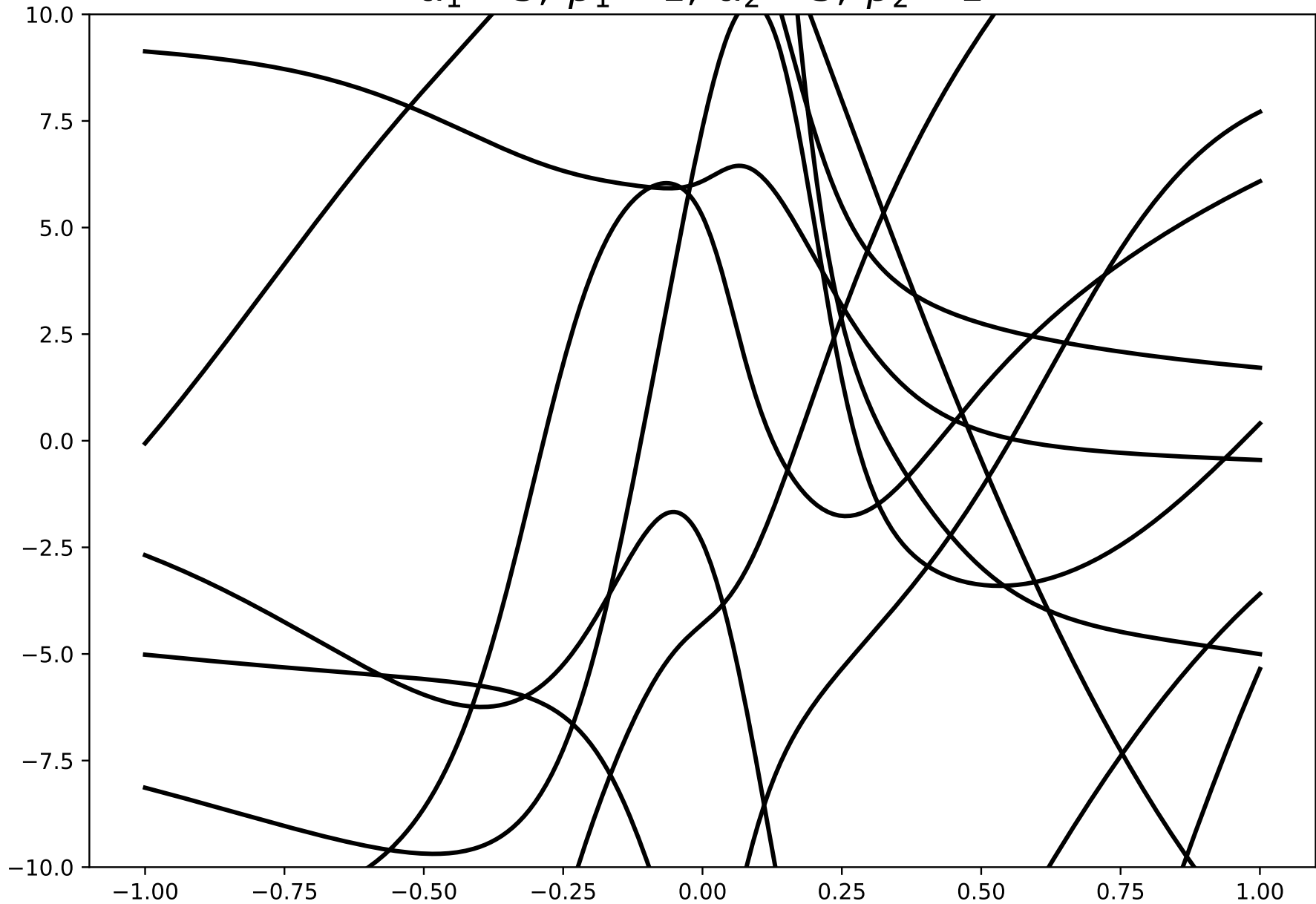
► Funciones generadas incrementando β_1 :

$$\alpha_1 = 5, \beta_1 = 5, \alpha_2 = 1, \beta_2 = 1$$



► Funciones generadas incrementando α_2 :

$$\alpha_1 = 5, \beta_1 = 1, \alpha_2 = 5, \beta_2 = 1$$



- **Con ReLUs en lugar de sigmoides:** si fijamos $\beta_\ell = 0$, la modificación de α_ℓ simplemente re-escala la salida:

$$f(\mathbf{x}; \boldsymbol{\alpha}, \boldsymbol{\beta} = \mathbf{0}) = \alpha_L \boldsymbol{\eta}_L(\cdots \varphi(\alpha_1 \boldsymbol{\eta}_1 \mathbf{x}) \cdots) \quad (122)$$

$$= \alpha_L \cdots \alpha_1 \boldsymbol{\eta}_L(\cdots \varphi(\boldsymbol{\eta}_1 \mathbf{x}) \cdots) \quad (123)$$

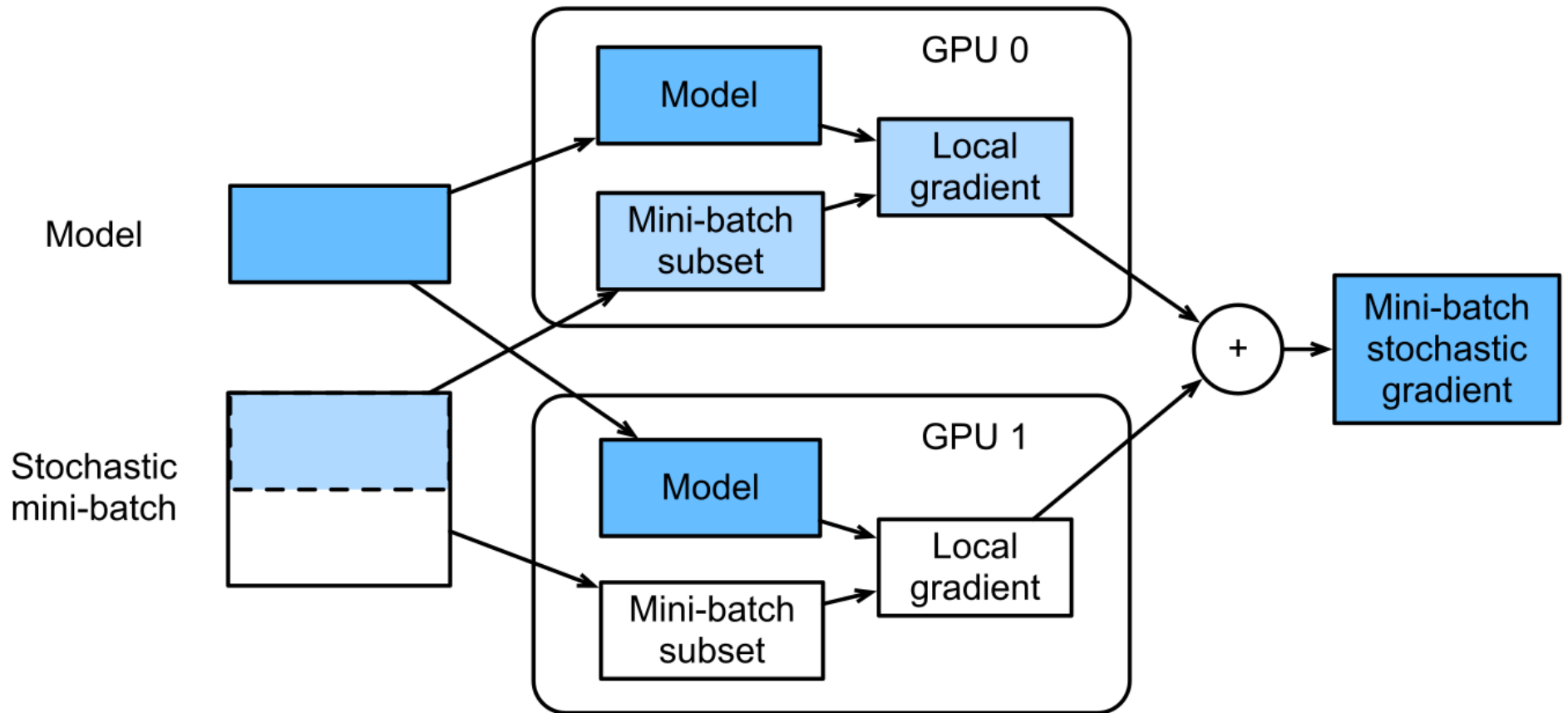
$$= \alpha_L \cdots \alpha_1 f(\mathbf{x}; (\boldsymbol{\alpha} = \mathbf{1}, \boldsymbol{\beta} = \mathbf{0})) \quad (124)$$

pues, en el caso de la ReLU, $\text{ReLU}(\alpha z) = \alpha \text{ReLU}(z)$ para $\alpha > 0$, y $\text{ReLU}(\alpha z) = 0$ para $\alpha < 0$ (ya que la pre-activación $z \geq 0$)

- En realidad, es el ratio entre α y β el que determina el comportamiento del modelo
- **Neural network Gaussian process:** dado que distribuciones diferentes sobre los parámetros corresponden a distribuciones diferentes sobre funciones, la inicialización del modelo puede verse como un muestreo de un punto sobre el prior; en el límite de redes infinitamente anchas, el prior puede derivarse analíticamente

13.4.6. Entrenamiento paralelo

- ▶ **Graphics processing units (GPUs):** muy eficientes para multiplicar matrices; actualmente se emplean múltiples GPUs para entrenar DNNs con grandes conjuntos de datos
- ▶ **Paralelismo de modelo:** partimos el modelo entre diferentes máquinas; en general resulta demasiado complicado
- ▶ **Paralelismo de datos:** partimos los datos entre diferentes máquinas, cada una con una copia del modelo; paso t de entrenamiento:
 1. Partimos el minibatch en las K máquinas para obtener $\mathcal{D}_t^k$
 2. Cada máquina k calcula su propio gradiente, $\mathbf{g}_t^k = \nabla_{\boldsymbol{\theta}} \mathcal{L}(\boldsymbol{\theta}; \mathcal{D}_t^k)$
 3. Recuperamos todos los gradientes locales en una máquina central (p.e., dispositivo 0) y los sumamos, $\mathbf{g}_t = \sum_{k=1}^K \mathbf{g}_t^k$
 4. Difundimos el gradiente sumado a todas las máquinas, $\tilde{\mathbf{g}}_t^k = \mathbf{g}_t$
 5. Cada máquina actualiza su copia de parámetros, $\boldsymbol{\theta}_t^k = \boldsymbol{\theta}_t^k - \eta_t \tilde{\mathbf{g}}_t^k$



- **all-reduce:** paso atómico que combina los pasos 3 y 4: recuperación de gradientes locales, suma y difusión a todas las máquinas
- ▷ **Entrenamiento síncrono:** cada máquina se bloquea hasta tener el sumado; como entrenar con una máquina (sin batchnorm)
- ▷ **Entrenamiento asíncrono** o **hogwild (glotón salvaje):** cada máquina actualiza su gradiente local (y parámetros distintos)

13.5. Regularización

13.5.1. Terminación temprana

- El método más sencillo para evitar el sobre-entrenamiento

13.5.2. Penalización de pesos

- Es usual emplear estimación MAP con priors Gaussianos para pesos, $\mathcal{N}(\boldsymbol{w} \mid \mathbf{0}, \alpha^2 \mathbf{I})$, y sesgos, $\mathcal{N}(\boldsymbol{b} \mid \mathbf{0}, \beta^2 \mathbf{I})$

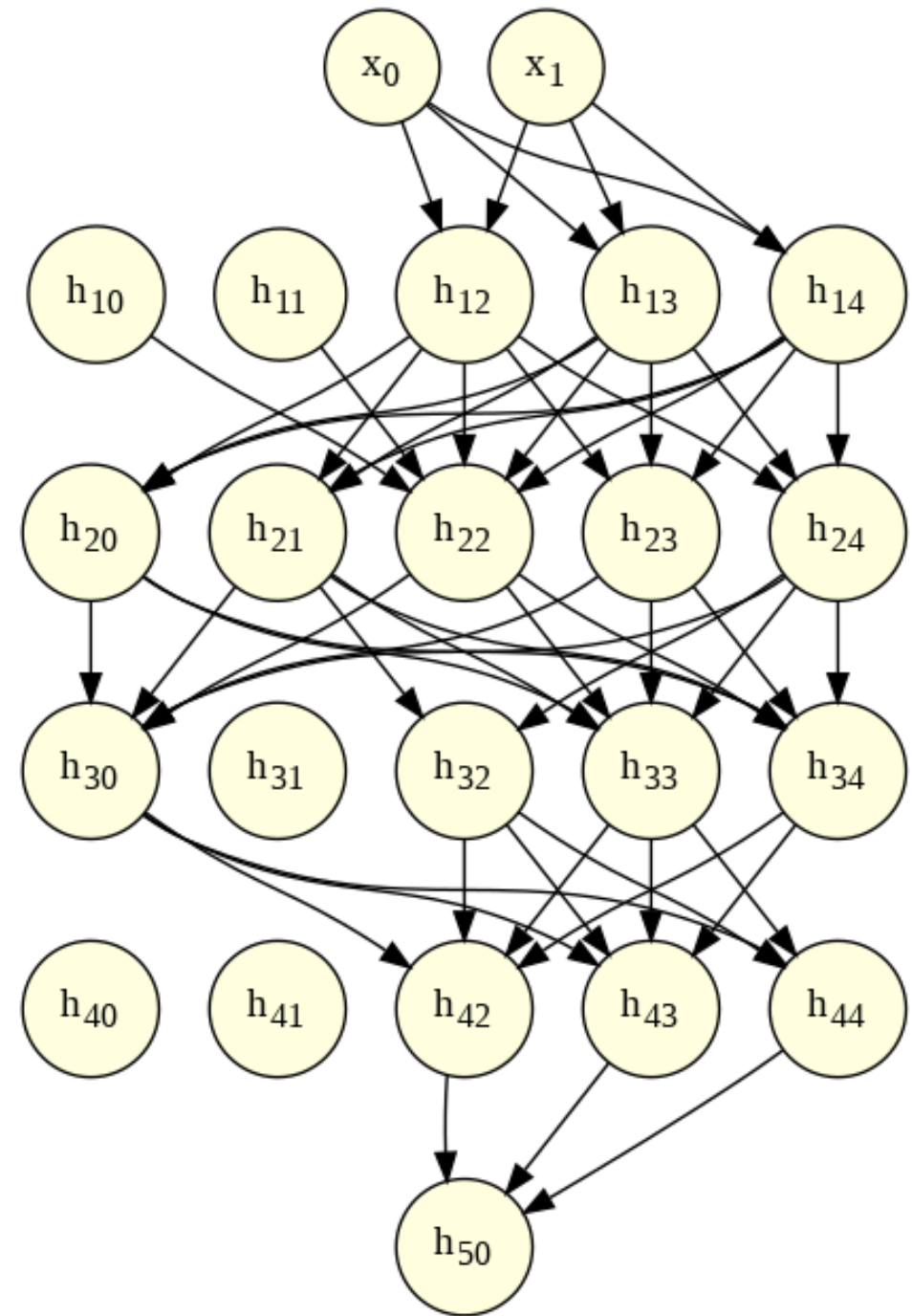
13.5.3. DNNs dispersas

- *Técnicas para obtener DNNs dispersas:* regularización ℓ_1 , automatic relevancy determination (ARD) y otros métodos

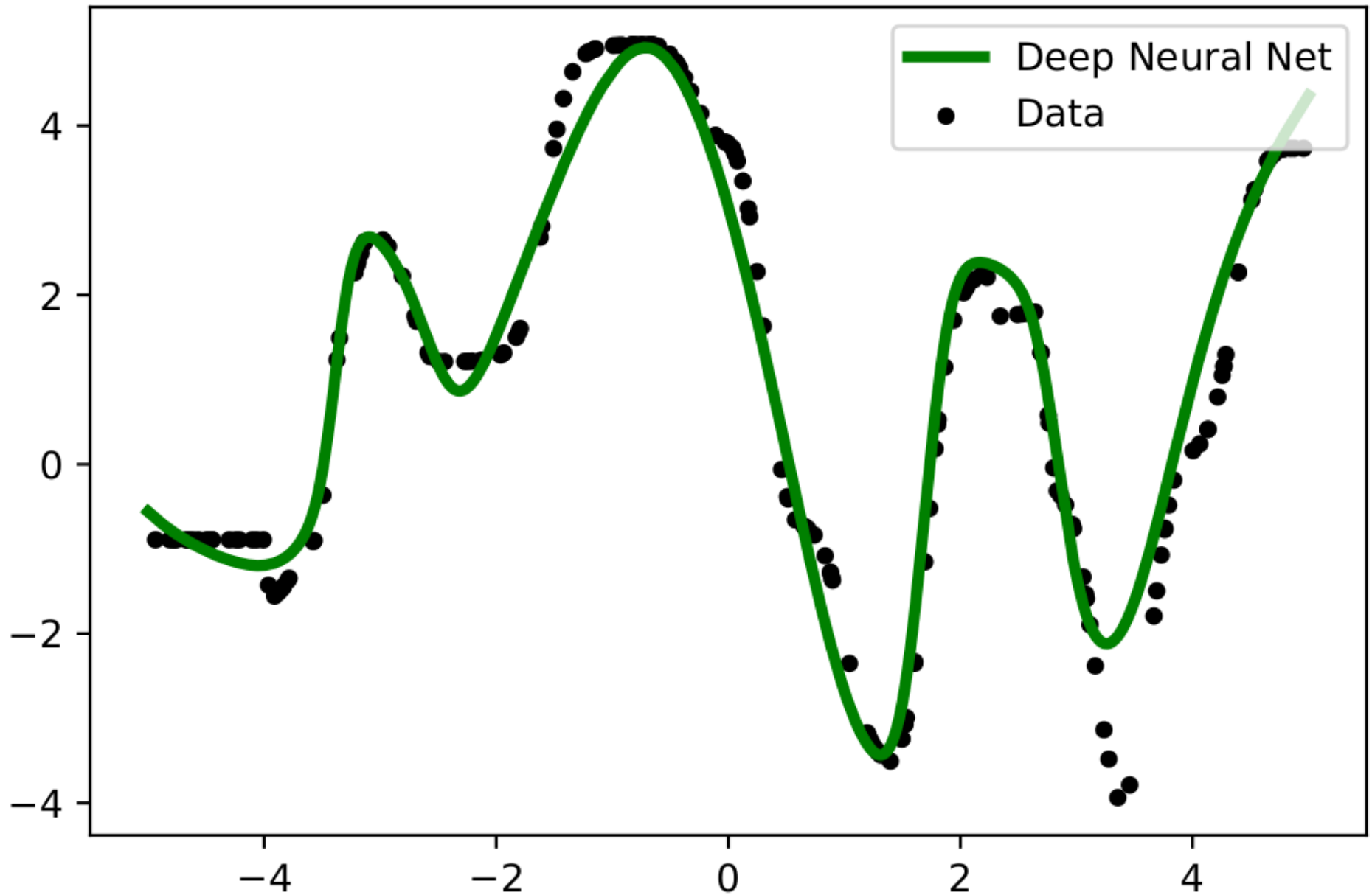
► *Ejemplo:*

MLP de 5 capas para ajustar datos 1d de regresión usando un regularizador ℓ_1 de pesos con el fin de obtener una topología dispersa

v Los nodos terminados en 0 producen sesgos



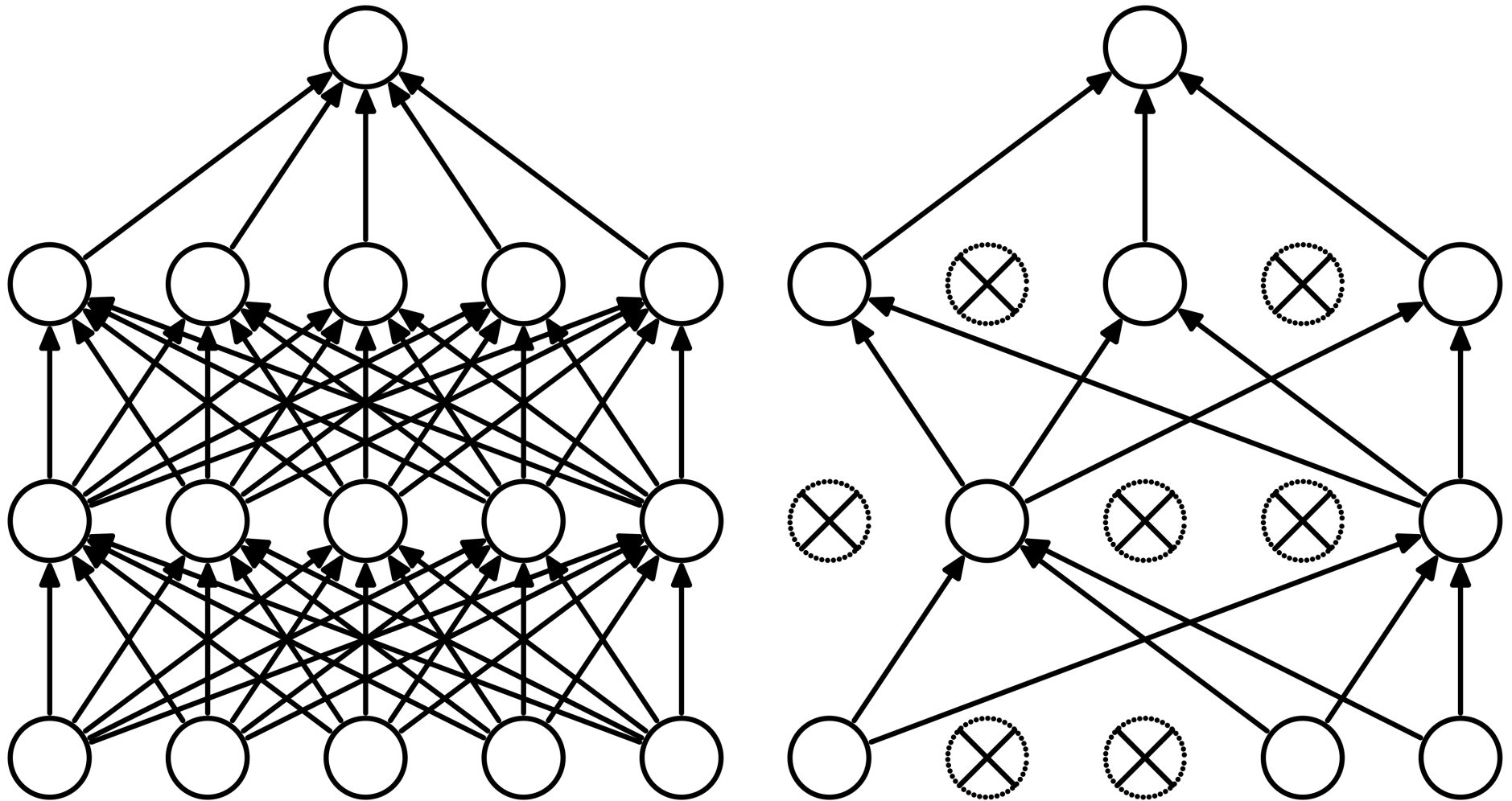
- *Ejemplo (cont.):* predicciones sobre datos de entrenamiento.



- ▶ ***Poco usadas en la práctica:*** las técnicas para obtener DNNs dispersas no se usan mucho ya que las GPUs están pensadas para multiplicación de matrices densa, por lo que el uso de matrices dispersas aporta poco ahorro computacional
- ▶ ***Dispersión grupal por bloques:*** sí se han obtenido mejoras computacionales importantes mediante el uso de matrices con dispersión grupal por bloques que permiten podar capas enteras

13.5.4. Dropout

- **Dropout:** apagado aleatorio de cada neurona con probabilidad p
- *Ejemplo:* MLP de 2 capas y dropout con probabilidad 0.5



- **Efectividad y popularidad para evitar sobre-entrenamiento:** dropout es una técnica muy efectiva y popular para evitar sobre-entrenamiento; intuitivamente, evita la co-adaptación compleja de unidades ocultas, esto es, cada unidad debe aprender a rendir bien incluso si otras unidades se apagan aleatoriamente
- **dropout como estimador de pesos ruidosos:** $\theta_{lij} = w_{lij}\epsilon_{li}$ donde $\epsilon_{li} \sim \text{Ber}(1 - p)$ es el término de ruido
 - ▷ Si muestreamos $\epsilon_{li} = 0$, todos los pesos que salen de la unidad i en la capa $l - 1$ hacia cualquier j en la l se fijan a 0
 - ▷ En test no suele usarse ruido, pero debería usarse $w_{lij} = \theta_{lij} \mathbb{E}[\epsilon_{li}]$ por consistencia con el entrenamiento; $\mathbb{E}[\epsilon] = 1 - p$ con Bernoulli
- **Monte Carlo dropout:** en test resulta un ensamble de redes

$$p(\mathbf{y} \mid \mathbf{x}, \mathcal{D}) \approx \frac{1}{S} \sum_{s=1}^S p(\mathbf{y} \mid \mathbf{x}, \hat{\mathbf{W}}\epsilon^s + \hat{\mathbf{b}}) \quad (125)$$

donde S es el número de muestras y $\hat{\mathbf{W}}\epsilon^s$ indica que multiplicamos las matrices de pesos estimadas por un vector ruido muestreado

13.5.5. Redes neuronales Bayesianas

- ▶ Las DNNs modernas se suelen entrenar mediante máxima verosimilitud (penalizada) para hallar un único ajuste de parámetros. Sin embargo, los modelos usuales tienen muchos más parámetros que puntos de datos, por lo que es de esperar que haya múltiples modelos que, ajustándose igual de bien que el entrenado, generalicen de formas diferentes
- ▶ **Bayesian neural network (BNN):** una manera de capturar la incertidumbre inducida consiste en marginalizar parámetros mediante la **distribución predictiva a posteriori**

$$p(\mathbf{y} \mid \mathbf{x}, \mathcal{D}) = \int p(\mathbf{y} \mid \mathbf{x}, \boldsymbol{\theta}) p(\boldsymbol{\theta} \mid \mathcal{D}) d\boldsymbol{\theta} \quad (126)$$

Este resultado puede interpretarse como un ensamble infinito de redes ponderadas diferentemente. La marginalización de parámetros permite evitar el sobre-entrenamiento, si bien es muy costosa